

Curso MLOps / DataOps

Manual del alumno · 2 días · paso a paso

José Picón · ANBAN

2026

- [Cómo usar este manual](#)
 - [Estructura](#)
 - [Cómo seguir la clase con este manual](#)
 - [Antes de empezar: comprobaciones](#)
 - [Convenciones tipográficas](#)
- [Instalación del entorno por sistema operativo](#)
 - [A · Windows 10/11](#)
 - [B · macOS \(Intel y Apple Silicon\)](#)
 - [C · Linux \(Ubuntu/Debian, Fedora, Arch\)](#)
 - [D · Comprobación final \(los tres sistemas\)](#)
 - [E · Levanta el entorno del curso](#)
 - [F · Crear un entorno virtual Python para los labs](#)
- [Conceptos clave con ejemplos](#)
 - [1 · DVC — versionado de datos](#)
 - [2 · Great Expectations — calidad de datos como código](#)
 - [3 · MLflow — experiment tracking](#)
 - [4 · FastAPI + Docker — servir el modelo](#)
 - [5 · GitHub Actions — CI/CD para ML](#)
 - [6 · Evidently — detección de drift](#)
 - [7 · Feature Store y governance \(visión panorámica\)](#)
 - [8 · Cómo encaja todo: el ciclo completo](#)
- [DÍA 1 — DataOps y fundamentos de MLOps](#)
 - [Hoja de ruta del día](#)

- [Cómo seguir el día](#)
- [Lo que debes lograr al final del Día 1](#)
- [1. Por qué existe MLOps](#)
 - [1.1 Una analogía rápida](#)
- [2. DevOps, DataOps, MLOps, AIOps, LLMOps](#)
- [3. El triángulo Código–Datos–Modelo](#)
- [4. Ciclo de vida: CRISP-DM y CRISP-ML\(Q\)](#)
- [5. Niveles de madurez MLOps \(Google\)](#)
 - [Nivel 0 — Proceso manual](#)
 - [Nivel 1 — Pipeline ML automatizado](#)
 - [Nivel 2 — CI/CD completo](#)
- [6. Roles en un equipo MLOps](#)
- [7. Diagnóstico de madurez: ejercicio](#)
 - [Escenario A](#)
 - [Escenario B](#)
 - [Escenario C](#)
- [8. Vocabulario mínimo](#)
- [9. Para profundizar](#)
 - [1. DataOps en cuatro principios](#)
 - [2. Versionado de datos: por qué Git no basta](#)
 - [3. Cómo funciona DVC](#)
 - [4. Calidad de datos: cinco dimensiones](#)
 - [5. Orquestación: panorama](#)
 - [6. Lineage y catálogo](#)
 - [7. Lab 1. Pipeline reproducible con DVC y validación](#)
 - [Objetivo](#)
 - [Prerrequisitos](#)
 - [Dataset](#)
 - [Paso 1. Inicializar Git y DVC](#)
 - [Paso 2. Configurar el remoto MinIO](#)
 - [Paso 3. Trackear el dataset](#)
 - [Paso 4. La suite de validación](#)
 - [Paso 5. El pipeline DVC](#)
 - [Paso 6. Romper el dato a propósito](#)
 - [Paso 7. Cambiar un parámetro](#)
 - [Paso 8. Simular un compañero nuevo](#)
 - [Checklist de cierre](#)

- [Reto opcional](#)
- [8. Errores frecuentes y cómo resolverlos](#)
- [9. Para profundizar](#)
- [Lab 1 — DataOps con DVC](#)
 - [Qué vas a aprender](#)
 - [Cómo ejecutar el lab](#)
 - [Antes de empezar](#)
 - [Si algo falla](#)
 - [Estructura de este lab](#)
 - [Tras completar el lab](#)
- [1. El problema que resuelve el tracking](#)
- [2. Anatomía de MLflow](#)
- [3. Vocabulario clave](#)
- [4. Alternativas a MLflow](#)
- [5. Reproducibilidad estricta](#)
- [6. Lab 2. MLflow Tracking y Model Registry](#)
 - [Objetivo](#)
 - [Prerrequisitos](#)
 - [Variables de entorno](#)
 - [Paso 1. Repasar el código de entrenamiento](#)
 - [Paso 2. Entrenar tres modelos](#)
 - [Paso 3. Comparar runs en la UI](#)
 - [Paso 4. Registrar el modelo ganador](#)
 - [Paso 5. Promocionar a Staging](#)
 - [Paso 6. Cargar desde el Registry](#)
 - [Checklist de cierre](#)
 - [Reto opcional](#)
- [7. Errores frecuentes y cómo resolverlos](#)
- [8. Buenas prácticas que pagan](#)
- [9. Para profundizar](#)
- [Lab 2 — MLflow Tracking + Model Registry](#)
 - [Objetivo](#)
 - [Pre-requisitos](#)
 - [Variables de entorno](#)
 - [Paso 1 · Entrenar 3 modelos](#)
 - [Paso 2 · Comparar runs](#)
 - [Paso 3 · Registrar el modelo ganador](#)

- [Paso 4 · Promocionar a Staging](#)
- [Paso 5 · Cargar el modelo desde el Registry](#)
- [Comprobaciones de cierre](#)
- [Reto opcional](#)
- [DÍA 2 — MLOps en producción](#)
 - [Hoja de ruta del día](#)
 - [Prerrequisitos del Día 2](#)
 - [Lo que debes lograr al final del Día 2](#)
- [1. Patrones de serving](#)
- [2. Cómo se empaqueta un modelo](#)
- [3. Cuatro principios de diseño de la API](#)
- [4. Métricas de rendimiento](#)
- [5. Trade-offs típicos](#)
- [6. Lab 3. API REST en contenedor](#)
 - [Objetivo](#)
 - [Arquitectura](#)
 - [Prerrequisitos](#)
 - [Paso 1. Arrancar en local](#)
 - [Paso 2. Probar con curl](#)
 - [Paso 3. Métricas en formato Prometheus](#)
 - [Paso 4. Construir la imagen Docker](#)
 - [Paso 5. Lanzar el contenedor](#)
 - [Paso 6. Stress test con Locust](#)
 - [Checklist de cierre](#)
 - [Reto opcional](#)
- [7. Errores frecuentes y cómo resolverlos](#)
- [8. Para profundizar](#)
- [Lab 3 — FastAPI + Docker para servir el modelo](#)
 - [Objetivo](#)
 - [Arquitectura](#)
 - [Ejecución local \(sin Docker\)](#)
 - [Probar la API](#)
 - [Construir la imagen](#)
 - [Stress test con Locust](#)
 - [Comprobaciones de cierre](#)
 - [Reto opcional](#)

- [1. CI, CD y CT](#)
 - [El triple bucle ML](#)
- [2. Un pipeline CI/CD típico en GitHub Actions](#)
- [3. Estrategias de despliegue](#)
- [4. Monitoring: dos familias de métricas](#)
- [5. Drift: tres sabores](#)
 - [Population Stability Index \(PSI\)](#)
- [6. Herramientas de drift](#)
- [7. Lab 4. CI/CD con GitHub Actions y drift con Evidently](#)
 - [Objetivo](#)
 - [Estructura](#)
 - [Paso 1. Repasar el workflow](#)
 - [Paso 2. Promoción condicional en local](#)
 - [Paso 3. Generar un dataset con drift](#)
 - [Paso 4. Generar el informe de drift](#)
 - [Paso 5. Endpoint /monitor en la API](#)
 - [Checklist de cierre](#)
 - [Reto opcional](#)
- [8. Errores frecuentes y cómo resolverlos](#)
- [9. Para profundizar](#)
- [Lab 4 — CI/CD con GitHub Actions + Monitoring con Evidently](#)
 - [Objetivo](#)
 - [Estructura](#)
 - [Paso 1 · Workflow GitHub Actions](#)
 - [Paso 2 · Probar el promote local](#)
 - [Paso 3 · Generar drift sintético y reporte](#)
 - [Paso 4 · Endpoint /monitor](#)
 - [Comprobaciones de cierre](#)
 - [Reto opcional](#)
- [1. Feature Store: cuándo te hace falta](#)
- [2. Model Governance](#)
 - [2.1 Model Card](#)
 - [2.2 Audit trail](#)
 - [2.3 RGPD y AI Act](#)
 - [2.4 Sesgo y fairness](#)
 - [2.5 Decisión humana](#)

- [3. Reproducibilidad estricta](#)
- [4. Cost y FinOps en ML](#)
- [5. LLMOps en un vistazo](#)
- [6. Lab 5. Recorrido end-to-end y governance](#)
 - [Objetivo](#)
 - [El recorrido en 8 comandos](#)
 - [Rellenar la Model Card](#)
 - [Checklist de governance](#)
 - [Discusión](#)
- [7. Hoja de ruta de aprendizaje](#)
- [8. Bibliografía y recursos](#)
 - [Libros](#)
 - [Webs](#)
 - [Whitepapers](#)
- [9. Entregable opcional del curso](#)
- [Lab 5 — Caso end-to-end + Governance](#)
 - [Objetivo](#)
 - [El recorrido en 8 comandos](#)
 - [Checklist de governance \(rellenar en grupo\)](#)
 - [Discusión guiada \(15 min\)](#)
 - [Entregable del curso](#)
- [Guía paso a paso de los labs con explicaciones](#)
 - [Lab 1 — DataOps con DVC + Great Expectations](#)
 - [Lab 2 — MLflow Tracking + Model Registry](#)
 - [Lab 3 — Servir el modelo con FastAPI + Docker](#)
 - [Lab 4 — CI/CD con GitHub Actions + Monitoring con Evidently](#)
 - [Lab 5 — Caso integrador end-to-end + Governance](#)
- [Stack del curso](#)
- [URLs y credenciales](#)
- [Variables de entorno \(ponlas en cada terminal nueva\)](#)
- [DVC](#)
- [Git + DVC](#)
- [MLflow](#)
 - [Desde Python](#)
 - [Model Registry](#)
- [FastAPI + Docker](#)

- [Probar la API con curl](#)
- [Locust \(stress test\)](#)
- [Evidently y drift](#)
- [Promote condicional](#)
- [Atajos para depuración](#)
- [Si algo se rompe sin remedio](#)
- [Errores frecuentes y cómo resolverlos](#)
 - [1 · Docker no arranca o no responde](#)
 - [2 · Puertos ocupados](#)
 - [3 · Postgres: “esta página no funciona” en el navegador](#)
 - [4 · localhost vs nombres de servicio Docker](#)
 - [5 · Errores de Python y entornos virtuales](#)
 - [6 · DVC: errores frecuentes](#)
 - [7 · Jupyter](#)
 - [8 · MLflow no responde o se queda colgado](#)
 - [9 · Comandos universales para diagnosticar](#)
 - [10 · Cuándo pedir ayuda](#)

Cómo usar este manual

Este cuadernillo recoge **todo el curso en un único PDF** organizado por días. Cada día encadena teoría, demos y laboratorios en el orden exacto en el que se imparten en el aula. La idea es que puedas seguir la sesión sin perderte y, después de clase, repasar o reproducir cualquier paso por tu cuenta.

Estructura

- **Instalación del entorno por sistema operativo** (Windows, macOS, Linux).
- **Conceptos clave con ejemplos** — qué hace DVC, MLflow, FastAPI, Evidently y los demás, con un ejemplo concreto y una analogía. Lectura previa muy recomendable.
- **Día 1 — DataOps y fundamentos de MLOps** (Módulos 1–3 + Labs 1–2).
- **Día 2 — MLOps en producción** (Módulos 4–6 + Labs 3–5).
- **Guía paso a paso de los labs con explicaciones** — cada fase de cada lab explicada en prosa: qué hace el comando, por qué lo necesitas, qué deberías ver.
- **Cheatsheet de comandos** — referencia rápida para los labs.
- **Errores frecuentes y soluciones** — diez secciones cubriendo los problemas que aparecen casi en cada edición.

Cómo seguir la clase con este manual

1. Empieza por la introducción del día (**DÍA 1** o **DÍA 2**) para ver la hoja de ruta de las 4 horas.

2. Cada **módulo** se compone de:

- Objetivos al inicio.
- Teoría con apoyo de slides (referencia al rango de slides del PPTX).
- Discusión y preguntas guía.

3. Cada **lab** trae:

- Prerrequisitos.
- Pasos numerados que reproducen lo que el profesor hace en pantalla.
- Comprobaciones (`Comprueba que...`) para validar tu progreso.
- Reto opcional al final.

Antes de empezar: comprobaciones

Asegúrate de tener el entorno listo:

```
cd docker
docker compose up -d
docker compose ps
```

Los cinco servicios deben aparecer `up` (Jupyter, MLflow, MinIO, Postgres, minio-init).

Accesos:

Servicio	URL	Credenciales
Jupyter	http://localhost:8888	token <code>anban</code>
MLflow	http://localhost:5050	—
MinIO console	http://localhost:9001	<code>minio</code> / <code>minio12345</code>
PostgreSQL	localhost:5432	<code>mlflow</code> / <code>mlflow</code> (db <code>mlflow</code>)

Si algún puerto está ocupado, edita el lado izquierdo del `:` en `docker/docker-compose.yml`.

Convenciones tipográficas

- `código entre comillas`: comando que escribirás en terminal o en una celda de Jupyter.
- **Negrita**: concepto clave que se va a desarrollar.
- *Bloque citado: tip, advertencia o referencia rápida.*
- *Cursiva*: término técnico en inglés, librería o producto.

Si tu entorno aún no está montado, empieza por **Instalación del entorno por sistema operativo**. Si ya tienes Docker y los servicios arriba, salta directo al **Día 1**. Si algo falla durante el curso, ve al apartado **Errores frecuentes y soluciones** del final.

Instalación del entorno por sistema operativo

Este capítulo te lleva paso a paso desde “máquina recién encendida” hasta “entorno del curso funcionando”. Las pantallas y comandos cambian según el sistema operativo. Salta al apartado que te corresponda.

*Resumen para los tres sistemas: necesitas **Docker**, **Git** y **Python 3.11+**. Todo lo demás (MLflow, MinIO, Postgres, Jupyter) lo levantará Docker por ti.*

A · Windows 10/11

A.1 Habilita WSL2 (subsistema Linux)

Docker Desktop en Windows funciona sobre WSL2. Abre **PowerShell como administrador** y ejecuta:

```
wsl --install
```

Reinicia el equipo. Tras el reinicio terminará la instalación y te pedirá un usuario y contraseña de Linux (Ubuntu por defecto). Anótalos.

Si `wsl --install` te dice que ya está instalado, perfecto: pasa al siguiente paso.

A.2 Instala Docker Desktop

1. Descarga el instalador desde <https://www.docker.com/products/docker-desktop/>.
2. Durante la instalación deja marcada la opción “**Use WSL 2 instead of Hyper-V**”.
3. Termina la instalación y arranca Docker Desktop. Espera a ver el icono de la ballena en la bandeja del sistema.
4. En *Settings* → *Resources* → *WSL Integration*, activa la integración con tu distribución (Ubuntu).

Comprobación:

```
docker --version  
docker compose version
```

Ambos deben mostrar un número de versión.

A.3 Instala Git

```
winget install --id Git.Git -e
```

O descarga el `.exe` desde <https://git-scm.com/download/win> y siguiente, siguiente, finalizar.

A.4 Instala Python 3.11

```
winget install --id Python.Python.3.11 -e
```

Marca **Add Python to PATH** si el instalador lo pregunta. Comprueba:

```
python --version  
pip --version
```

A.5 Elige tu terminal

Vas a trabajar mucho en línea de comandos. Te recomendamos uno de estos dos, según prefieras:

- **PowerShell** (viene de serie). Funciona, pero algunos comandos del curso usan sintaxis Unix.
- **WSL/Ubuntu** (terminal Linux dentro de Windows). Es lo más compatible con el material del curso. Abre el menú Inicio y busca “Ubuntu”.

*Si eliges WSL, ejecuta todo desde dentro de Ubuntu, **no desde PowerShell**, y guarda el repo del curso en el sistema de archivos de Linux (por ejemplo `~/Anbam-alumno`), no en `/mnt/c/...` — el rendimiento de Docker se desploma si los volúmenes apuntan a `C:`.*

B · macOS (Intel y Apple Silicon)

B.1 Instala Homebrew

Si aún no lo tienes:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Al terminar, el propio instalador te indicará dos comandos para añadir Homebrew al PATH. Cópialos y ejecútalos.

B.2 Instala Docker Desktop

```
brew install --cask docker
```

O descarga el `.dmg` desde `https://www.docker.com/products/docker-desktop/`. Asegúrate de elegir la versión correcta: **Apple Silicon (M1/M2/M3/M4)** o **Intel**.

Abre Docker Desktop al menos una vez para que conceda permisos. El icono de la ballena debe aparecer en la barra superior.

Comprobación:

```
docker --version
docker compose version
```

B.3 Instala Git y Python

```
brew install git python@3.11
```

Comprueba:

```
git --version
python3 --version
```

En macOS, el binario es `python3` y `pip3`. Cuando el material diga `python` o `pip`, usa `python3` y `pip3` (o crea un alias).

B.4 Aviso para Apple Silicon

Algunas imágenes Docker antiguas no tienen build `arm64`. En ese caso Docker te avisará y usará emulación `x86_64` (más lenta). Las imágenes del curso (postgres, minio, jupyter, mlflow) sí tienen builds nativos `arm64`, así que no deberías tener problema.

C · Linux (Ubuntu/Debian, Fedora, Arch)

C.1 Instala Docker Engine + Compose

Ubuntu / Debian

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl enable --now docker
```

Fedora

```
sudo dnf install -y docker docker-compose-plugin
sudo systemctl enable --now docker
```

Arch

```
sudo pacman -S --noconfirm docker docker-compose
sudo systemctl enable --now docker
```

C.2 Permite ejecutar Docker sin sudo

```
sudo usermod -aG docker $USER
newgrp docker
```

Si abres una terminal nueva, recuerda hacer `newgrp docker` o cerrar/abrir sesión completa para que el grupo entre en vigor.

C.3 Comprueba

```
docker --version
docker compose version
docker run --rm hello-world
```

El último comando descarga una imagen de prueba y la ejecuta; si ves “Hello from Docker!” está todo bien.

C.4 Git y Python

Ubuntu / Debian

```
sudo apt install -y git python3 python3-pip python3-venv
```

Fedora

```
sudo dnf install -y git python3 python3-pip
```

Arch

```
sudo pacman -S --noconfirm git python python-pip
```

D · Comprobación final (los tres sistemas)

Abre una terminal y ejecuta:

```
docker --version
docker compose version
```

```
git --version
python --version || python3 --version
```

Los cuatro deben devolver un número de versión sin errores.

E · Levanta el entorno del curso

1. Descomprime `Anbam-alumno.zip` (o clona el repo) en una carpeta cómoda:

- **Windows + WSL:** `~/Anbam-alumno` (dentro de Ubuntu, **no** en `/mnt/c/`).
- **Windows nativo:** `C:\Users\<tu-usuario>\Anbam-alumno`.
- **macOS:** `~/Anbam-alumno`.
- **Linux:** `~/Anbam-alumno`.

2. Entra en la subcarpeta `docker` y levanta el stack:

```
cd Anbam-alumno/docker
docker compose up -d
```

La primera vez tarda unos minutos (descarga ~2 GB).

3. Comprueba que los cinco servicios están corriendo:

```
docker compose ps
```

Debes ver `Up (healthy)` o `Up` en: `anban-postgres`, `anban-minio`, `anban-mlflow`, `anban-jupyter`. El `anban-minio-init` aparece como `Exited (0)` y eso es correcto: es un contenedor de un solo uso que crea los buckets y termina.

4. Abre en el navegador:

- Jupyter: `http://localhost:8888` (token: `anban`).
- MLflow: `http://localhost:5050`.
- MinIO console: `http://localhost:9001` (`minio` / `minio12345`).

Importante: la consola de MinIO está en **9001** (web HTML para humanos). La API S3 que usan DVC y MLflow está en **9000** y no se navega; es protocolo binario.

F · Crear un entorno virtual Python para los labs

Algunos labs instalan paquetes locales (DVC, FastAPI, etc.). Crea un venv dedicado para no contaminar el sistema:

Windows (PowerShell)

```
cd Anbam-alumno
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -U pip
pip install -r requirements.txt
```

Si te dice “ejecución de scripts deshabilitada”, ejecuta en PowerShell como admin:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned .
```

macOS / Linux / WSL

```
cd Anbam-alumno
python3 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -r requirements.txt
```

Cuando veas el prefijo `(.venv)` en tu prompt, el venv está activo. Para desactivarlo: `deactivate` .

*Para los labs ejecuta los comandos **con el venv activo**, así los binarios (`dvc` , `mlflow` , `uvicorn`) usan los paquetes correctos.*

Conceptos clave con ejemplos

Antes de empezar con los labs conviene tener una imagen clara de **qué hace cada herramienta y por qué la usamos**. Esta sección recorre uno a uno los protagonistas del curso (DVC, Great Expectations, MLflow, FastAPI, Docker, GitHub Actions, Evidently...) con un ejemplo concreto y una analogía. Cuando llegues al lab correspondiente sabrás no solo el comando, sino lo que está pasando por debajo.

Léelo de corrido si tienes 30 minutos antes de la primera sesión. Vuelve aquí cuando un comando no te haga clic.

1 · DVC — versionado de datos

El problema

Tienes un proyecto con un dataset de 2 GB. Si lo metes en Git:

- Cada clon descarga 2 GB.
- Cambias una fila → Git guarda los 2 GB enteros otra vez (no sabe hacer diff de un CSV).
- GitHub te bloquea: el límite por archivo son 100 MB.
- Te das cuenta de que tu modelo cambia los resultados según la versión del CSV, pero no puedes saber **con qué versión exacta** entrenaste un modelo de hace un mes.

Lo que hace DVC

DVC versiona los datos **fuera** de Git, dejando en Git solo un puntero (un archivo `.dvc` de 100 bytes) con el hash MD5 del archivo real.

Analogía: imagina que Git es tu agenda. En lugar de pegar el documento entero (datos), pegas una pegatina con su número de archivador. El documento real está en el archivador (MinIO/S3). Si alguien quiere consultarlo, lee la pegatina y va al archivador.

Ejemplo en 4 comandos

Partimos de un repo Git con un CSV de 2 GB:

```
git init
dvc init                # crea .dvc/ con configuración
dvc add data/big.csv    # genera data/big.csv.dvc (el "puntero")
git add data/big.csv.dvc .gitignore
git commit -m "track big.csv with dvc"
```

Lo que pasa por dentro:

1. DVC calcula `md5("contenido del CSV") = 8e7c41...`.
2. Mueve el CSV a `.dvc/cache/8e/7c41...` (un cache local indexado por hash).
3. Crea un enlace duro a esa cache desde `data/big.csv`.
4. Escribe `data/big.csv.dvc` con el hash → eso es lo único que va a Git.

Para compartirlo con tu equipo necesitas un **remote** (S3, MinIO, GCS...):

```
dvc remote add -d minio s3://datasets/mi-proyecto
dvc remote modify minio endpointurl http://localhost:9000
dvc push                                # sube el blob real al remote
```

Tu compañero:

```
git clone <repo>
dvc pull                                # baja los blobs reales según los .dvc
```

Por qué versionar también el pipeline

`dvc.yaml` describe cómo se generan los datos derivados:

```
stages:
  preprocess:
    cmd: python src/preprocess.py
    deps:
      - data/raw/adult.csv
      - src/preprocess.py
    outs:
      - data/processed/train.parquet
```

Cuando ejecutas `dvc repro`:

- DVC compara hashes de las dependencias con la última vez.
- Si no han cambiado, **no ejecuta nada** (cache hit).
- Si han cambiado, ejecuta `cmd` y regenera los `outs`.

Resultado: **reproducibilidad exacta** y velocidad. Cambias el código del preprocesado y solo se regenera lo afectado, igual que `make` pero para pipelines de datos.

2 · Great Expectations — calidad de datos como código

El problema

Llega un nuevo lote de datos. Una columna que solía tener edades entre 17 y 90 ahora trae alguna fila con `age = -5` (bug en el sistema upstream). El pipeline entrena, no falla, y publicas un modelo que va a dar predicciones absurdas. Lo descubres dos semanas después.

Lo que hace Great Expectations

Te permite escribir “expectativas” (asserts inteligentes) sobre tu dataset:

```
import great_expectations as ge

df = ge.from_pandas(pd.read_csv("data/raw/adult.csv"))
df.expect_column_values_to_be_between("age", 17, 90)
df.expect_column_values_to_not_be_null("income")
df.expect_column_values_to_be_in_set("workclass", ["Private", "Self-emp", "Gov"])
```

Cada `expect_*` te devuelve un resultado con `success: true/false`. Las metes en el pipeline como gate: si una falla, el pipeline se para y suena la alarma.

*Analogía: es el **control de calidad** de una fábrica. Antes de que la pieza pase a la siguiente máquina, la inspeccionas con un calibre. Si no pasa, paras la línea.*

Las 5 dimensiones que vamos a validar

1. **Schema**: las columnas que esperas siguen ahí.
2. **Rango**: valores numéricos dentro de límites razonables.
3. **Dominio cerrado**: categorías dentro de un conjunto conocido.
4. **No-nulos**: las columnas críticas no traen huecos.
5. **Unicidad**: el identificador no se repite.

En el Lab 1 escribirás las cuatro primeras y romperás el CSV a propósito para ver cómo el pipeline falla seguro.

3 · MLflow — experiment tracking

El problema

Pasas tres días probando combinaciones: `n_estimators=100`, `n_estimators=200`, otro modelo, otro dataset, otro preprocesado. Al final tienes:

- 17 archivos `model_v2_final_FINAL_realFINAL.pkl`.
- Un Excel donde anotaste a mano algunas accuracies.
- Cero certeza sobre qué hiperparámetros usó el modelo bueno.

Lo que hace MLflow

MLflow registra cada **run** (entrenamiento) con: parámetros, métricas, código, modelo serializado y firma. Después puedes filtrar, comparar y promocionar.

Ejemplo mínimo

```
import mlflow
from sklearn.ensemble import RandomForestClassifier

mlflow.set_tracking_uri("http://localhost:5050") # tu servidor MLflow
mlflow.set_experiment("adult-income")

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)
    model = RandomForestClassifier(n_estimators=100, max_depth=5).fit(X_train,
y_train)

    acc = model.score(X_test, y_test)
    mlflow.log_metric("accuracy", acc)
    mlflow.sklearn.log_model(model, "model")
```

Tras ejecutarlo, abres `http://localhost:5050` y ves:

- El run que acabas de crear.
- Sus parámetros y métricas.
- El modelo `model/` adjunto, descargable.

Ejecuta el mismo script con `n_estimators=200`. Ahora tienes dos runs y puedes compararlos lado a lado en la UI.

*Analogía: un **cuaderno de laboratorio digital**. Cada experimento se firma, se fecha, queda registro de la receta exacta y de los resultados. Si seis meses después alguien dice “¿con qué entrenamos esto?”, la respuesta está en el cuaderno.*

Anatomía MLflow

Concepto	Qué es
Tracking	El servidor donde se registran los runs.
Experiment	Carpeta de runs relacionados (ej. “adult-income”).
Run	Una ejecución concreta con sus params y métricas.
Artifact	Cualquier archivo asociado al run (modelo, gráfica).
Registry	Catálogo central de modelos con etapas (None → Staging → Production → Archived).

Por qué Postgres + MinIO

MLflow necesita dos backends:

- **Postgres** (backend store): guarda metadatos — params, métricas, tags, runs.
- **MinIO** (artifact store): guarda los blobs grandes — modelos, gráficas, datos de muestra.

En el `docker-compose.yml` ya los tienes los dos cableados.

4 · FastAPI + Docker — servir el modelo

El problema

Tu modelo entrenado vive en `model.pkl` en tu portátil. La gente del producto pregunta: “¿cómo lo consume el resto del equipo? ¿desde móvil? ¿desde la web?”. Si la respuesta es “cárgalo en Python”, no tienes un sistema, tienes un script.

Lo que hace FastAPI

Convierte tu modelo en un **endpoint HTTP** al que cualquiera puede llamar:

```
from fastapi import FastAPI
from pydantic import BaseModel
import mlflow.pyfunc

app = FastAPI()
model = mlflow.pyfunc.load_model("models:/clf/Staging")

class Input(BaseModel):
    age: int
    workclass: str
    hours_per_week: int

@app.post("/predict")
def predict(payload: Input):
    pred = model.predict([[payload.age, payload.workclass, payload.hours_per_week]])

    return {"prediction": pred.tolist()}
```

Lanzas el servidor:

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Y desde otro terminal:

```
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"age": 39, "workclass": "Private", "hours_per_week": 40}'
```

Respuesta: `{"prediction": [1]}`.

*Analogía: pasas de tener un **chef cocinando en su cocina** a un **restaurante con ventanilla**: cualquiera pide por la ventanilla (HTTP) y recibe su plato (predicción). Más estandarizado, escalable y monitorizable.*

Por qué dockerizar el servicio

Sin Docker, para correr tu API alguien necesita: Python 3.11, las versiones exactas de `scikit-learn`, `mlflow`, las variables de entorno, etc. Con Docker entregas **una imagen** que ya trae todo dentro. La ejecutas en cualquier máquina con Docker y funciona.

Dockerfile típico:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Construyes y ejecutas:

```
docker build -t mi-modelo:1.0 .
docker run -p 8000:8000 mi-modelo:1.0
```

Pydantic: contrato fuerte en la entrada

`BaseModel` valida automáticamente cada petición. Si alguien manda `"age": "treinta y nueve"`, FastAPI rechaza con `422 Unprocessable Entity` antes de tocar el modelo. Eso convierte tu API en **autodefensiva**: no llegan datos sucios al modelo.

5 · GitHub Actions — CI/CD para ML

El problema

Tu compañero sube un cambio al código del modelo. ¿Está roto? ¿Funcionan los tests? ¿El nuevo modelo es mejor que el de producción? Si la respuesta a cualquiera de esas es “no me he enterado”, tienes un problema serio.

Lo que hace GitHub Actions

Define **pipelines** que se ejecutan automáticamente cuando ocurren eventos (push, pull request, cron). Cada paso se ejecuta en un contenedor limpio.

Ejemplo: pipeline ML típico

```
# .github/workflows/ml.yml
on: [push, pull_request]

jobs:
  test-and-train:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```

- uses: actions/setup-python@v5
  with: { python-version: '3.11' }
- run: pip install -r requirements.txt
- run: pytest # tests
- run: python src/train.py # entrena
- run: python src/evaluate.py # compara con baseline
- run: python src/promote.py # promociona si mejora

```

Cuando alguien hace `git push`, GitHub Actions:

1. Levanta un Ubuntu limpio.
2. Hace `git checkout` del código.
3. Instala dependencias.
4. Ejecuta los pasos en orden.
5. Marca el commit con  o  en la web de GitHub.

CI vs CD vs CT

- **CI** (Continuous Integration): cada cambio se valida automáticamente (tests, lint).
- **CD** (Continuous Deployment): si valida, se despliega.
- **CT** (Continuous Training): periódicamente se reentrena con datos nuevos.

ML necesita los tres bucles. CI y CD son los mismos de software; CT es exclusivo de ML porque el modelo **se desgasta con el tiempo** aunque el código no cambie (los datos del mundo cambian).

6 · Evidently — detección de drift

El problema

Entrenaste el modelo en datos de 2024. En 2026 las distribuciones han cambiado: la gente teletrabaja más, el rango de edades se desplaza, ciertas categorías aparecen que antes no existían. Tu modelo sigue funcionando técnicamente — devuelve un número — pero sus predicciones cada vez son más malas. Y no te das cuenta hasta que un cliente se queja.

Lo que hace Evidently

Compara dos datasets (referencia vs nuevo lote) y te dice si **las distribuciones se han movido**. Genera un informe HTML con tests estadísticos y visualizaciones.

Ejemplo

```

from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

reference = pd.read_csv("data/2024_q1.csv")
current   = pd.read_csv("data/2026_q1.csv")

report = Report(metrics=[DataDriftPreset()])

```

```
report.run(reference_data=reference, current_data=current)
report.save_html("drift_report.html")
```

Abres `drift_report.html` y ves columna a columna:

- ¿Ha cambiado la media de `age` ?
- ¿Ha aparecido un nuevo valor en `workclass` ?
- Tests: PSI, Kolmogorov-Smirnov, Wasserstein → cada uno con su umbral.

Tres tipos de drift que vamos a vigilar

Tipo	Qué cambia	Ejemplo
Data drift	$P(X)$	La edad media de los usuarios sube 5 años.
Concept drift	$P(y X)$	Mismas features dan otro resultado. Pandemia, regulación.
Label drift	$P(y)$	El porcentaje de clase positiva pasa del 20 % al 45 %.

Los tres degradan el modelo, pero por mecanismos distintos. Cada uno se mide de forma distinta.

*Analogía: tu modelo es un **traje hecho a medida** del cliente medio de 2024. En 2026 los cuerpos han cambiado un poco (data drift), el estilo que pide la gente es otro (concept drift) y la proporción de tallas que se vende ya no es la misma (label drift). El traje sigue siendo el mismo; el mundo no.*

7 · Feature Store y governance (visión panorámica)

Feature Store: por qué

Imagina que dos equipos usan la feature `customer_lifetime_value`. Uno la calcula en el entrenamiento con SQL en Snowflake. El otro la calcula en producción con Python sobre el último mes. Pequeñas diferencias en la definición → **el modelo entrenado no es el que ejecuta en producción** → predicciones malas.

Una **Feature Store** (Feast, Tecton, Hopsworks) centraliza definiciones de features con dos modos:

- **Offline store** (BigQuery, Snowflake, Parquet) para entrenamiento.
- **Online store** (Redis, DynamoDB) para serving con baja latencia.

Misma definición, dos materializaciones consistentes.

Governance: lo que pasa cuando ML se toma en serio

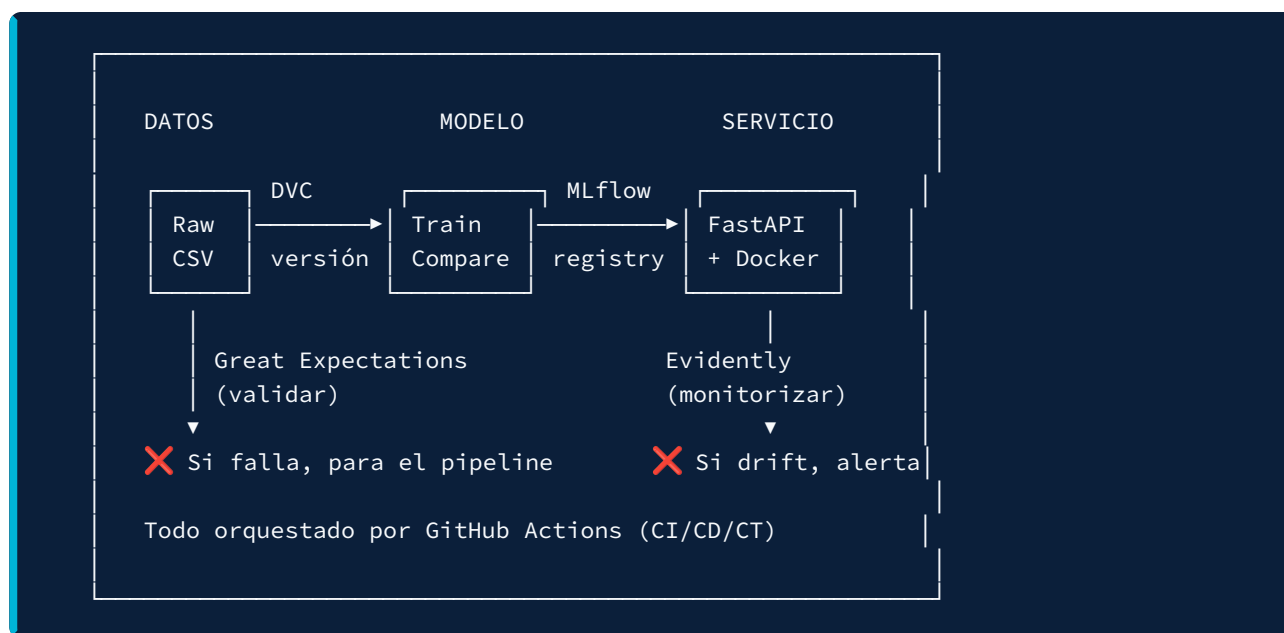
Cuando un modelo decide quién accede a un crédito o quién es priorizado en una lista de espera médica, ya no basta con “tiene buena accuracy”:

- **Model card**: documento que explica para qué sirve el modelo, sus límites, sus métricas por subgrupo (¿funciona igual para mayores que para jóvenes?).

- **Audit trail:** registro inmutable de qué datos y qué código produjeron qué modelo. MLflow + DVC + Git te lo dan.
- **AI Act / RGPD:** el modelo debe ser auditable. Si una persona pide explicación de una decisión, tienes que poder darla.
- **Fairness:** medir disparidad de error entre grupos protegidos.

En el Lab 5 cerraremos el ciclo y discutiremos qué falta para llevar todo esto a producción real.

8 · Cómo encaja todo: el ciclo completo



Resumen de qué herramienta tapa cada agujero:

- **Datos sin control** → DVC (versionado) + Great Expectations (validación).
- **Experimentos sin registro** → MLflow (tracking + registry).
- **Modelo aislado en un portátil** → FastAPI + Docker (servirlo como API).
- **Cambios sin validar y modelo que se desgasta** → GitHub Actions (CI/CD/CT) + Evidently (drift).
- **Caja negra ante auditoría** → Model card + audit trail + métricas por subgrupo.

Cuando avances por los labs, pregúntate en cada paso: “¿qué agujero estoy tapando con este comando?”. Esa pregunta es la que separa “ejecutar un tutorial” de “entender MLOps”.

DÍA 1 — DataOps y fundamentos de MLOps

Duración: 4 h (240 min) · **Carga:** teoría 50 % · práctica 50 %

Hoja de ruta del día

Hora aprox.	Bloque	Tipo
00:00 – 00:45	Módulo 1 — Introducción MLOps / DataOps	Teoría
00:45 – 01:35	Módulo 2 — DataOps (versionado, calidad)	Teoría
01:35 – 02:25	Lab 1 — DVC + Great Expectations	Práctica
02:25 – 02:35	Pausa	—
02:35 – 03:25	Módulo 3 — Experiment tracking (MLflow)	Teoría
03:25 – 04:00	Lab 2 — MLflow Tracking + Model Registry	Práctica

Cómo seguir el día

1. **Entorno arriba antes de empezar.** En la carpeta `docker/` ejecuta:

```
docker compose up -d
```

Comprueba con `docker compose ps` que los cinco servicios están `Up`.

2. **Material a mano.** Abre este PDF en una mitad de la pantalla y Jupyter (`http://localhost:8888` , token `anban`) en la otra.
3. **Anota dudas en el cuadernillo.** Cada módulo tiene preguntas guía al final que se discuten en pausa.
4. **Si te pierdes en un lab.** No bloques al grupo: salta al siguiente módulo y retoma el lab en la pausa o en casa con el README correspondiente.

Lo que debes lograr al final del Día 1

- Diagnosticar el nivel de madurez MLOps de un sistema ML real.
- Versionar un dataset con DVC y validarlo con Great Expectations.
- Trackear experimentos con MLflow y promocionar el mejor modelo a *Staging*.

1. Por qué existe MLOps

Llevas seis meses entrenando un modelo de clasificación. Lo entregas a producción en diciembre. En marzo, alguien te pregunta:

- ¿Con qué versión del dataset se entrenó?
- ¿Qué hiperparámetros tenía?
- ¿Cuántas predicciones lleva hechas?
- Si lo necesitamos reentrenar igual hoy, ¿podemos?

Si no tienes respuestas claras a estas cuatro preguntas, tu sistema sufre **deuda técnica oculta**, el problema descrito por Sculley y otros en el famoso paper de Google *Hidden Technical Debt in Machine Learning Systems* (2015). El paper demuestra con un esquema que el código del modelo es una fracción minúscula del sistema real, rodeado de servicios de validación de datos, configuración, monitoring, serving, ingestión, etc.

Idea central. El código del modelo es el 5% del sistema. El otro 95% es plomería. MLOps es la disciplina que se ocupa de esa plomería.

1.1 Una analogía rápida

Piensa en una receta de cocina. Para reproducir una cena de Navidad no basta con la receta: necesitas saber qué ingredientes usaste, de qué marca, cuánto pesaba la pieza de carne, qué temperatura tenía el horno *real*, en qué cocina lo hiciste. Si solo guardas la receta, la próxima vez sale distinto.

Con los modelos pasa igual. Guardar el script de entrenamiento es guardar la receta. Si no guardas el dataset exacto, los parámetros, la versión de las librerías y la máquina, **no estás guardando el modelo, estás guardando una promesa.**

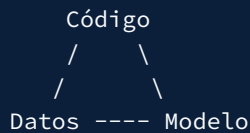
2. DevOps, DataOps, MLOps, AIOps, LLMOps

Mucha gente usa estos términos como sinónimos. No lo son.

Disciplina	Qué versiona	Qué automatiza
DevOps	Código	Build, test, deploy
DataOps	Datos y transformaciones	Pipelines de ingesta, validación, lineage
MLOps	Código, datos y modelo	Reentrenamiento, despliegue, monitorización
AIOps	Logs y métricas de infraestructura	Detección de anomalías en sistemas
LLMOps	Prompts, modelos, embeddings y RAG	Evaluación, guardrails, versionado de prompts

DataOps y MLOps son **complementarios**: DataOps termina cuando hay un dataset limpio y reproducible. MLOps empieza ahí. Sin DataOps, MLOps es un castillo de naipes.

3. El triángulo Código–Datos–Modelo



Los tres vértices están acoplados. Si cambia uno y no tocas los otros, el sistema deja de ser coherente:

- Cambias el código sin reentrenar → modelo desactualizado.
- Cambias el dato sin re-evaluar → puede haber degradación silenciosa.
- Cambias el modelo sin versionar el dato → no puedes reproducirlo.

Por eso necesitamos versionar los tres. Git resuelve el código. DVC, LakeFS o Delta Lake resuelven el dato. MLflow, Weights & Biases o similares resuelven el modelo y los experimentos.

4. Ciclo de vida: CRISP-DM y CRISP-ML(Q)

CRISP-DM es el ciclo clásico de minería de datos (1996, todavía vigente):

1. Entendimiento del negocio.
2. Entendimiento de los datos.
3. Preparación de los datos.
4. Modelado.
5. Evaluación.
6. Despliegue.

CRISP-ML(Q) es una revisión moderna (Studer et al., 2021) que añade explícitamente:

- **Calidad de datos** como fase propia y continua.
- **Monitorización** posterior al despliegue.
- **Mantenimiento**: drift, retraining, retirada del modelo.

En la práctica, MLOps implementa CRISP-ML(Q) con herramientas concretas.

5. Niveles de madurez MLOps (Google)

Google publicó un whitepaper muy conocido (*MLOps: Continuous delivery and automation pipelines in machine learning*) con tres niveles de madurez. Es la forma más extendida de diagnosticar dónde está una organización.

Nivel 0 — Proceso manual

- El data scientist trabaja en un notebook.
- Entrega un fichero `.pkl` o lo despliega manualmente.
- No hay tests automáticos del modelo.
- No hay tracking de experimentos.
- Si algo falla, se rehace a mano.

La mayoría de organizaciones está aquí, aunque alguna no lo admita.

Nivel 1 — Pipeline ML automatizado

- El entrenamiento es un pipeline reproducible, no un notebook.
- Hay validación de datos automática.
- Hay un registry de modelos.
- El despliegue está automatizado.
- Existe monitorización básica.

Llegar aquí ya supone un cambio cultural y técnico importante.

Nivel 2 — CI/CD completo

- Cada commit puede disparar entrenamiento, evaluación, promoción y despliegue.
- Reentrenamiento automático ante drift.
- Despliegues canary o blue-green.
- Observabilidad completa con alerting.
- Rollback automático.

Pocas organizaciones tienen este nivel completo, y casi siempre solo para sus modelos críticos.

6. Roles en un equipo MLOps

Rol	Responsabilidad principal
Data Engineer	Pipelines de datos, calidad, almacenamiento
Data Scientist	Modelado, experimentación, feature engineering
ML Engineer	Productivización del modelo, código de calidad
MLOps Engineer	Plataforma, automatización, observabilidad
SRE / ML Platform	Infraestructura, escalado, fiabilidad

En equipos pequeños, una sola persona cubre varios roles. En equipos grandes, hay especialización clara. Lo importante es que **alguien** se haga cargo de cada función.

7. Diagnóstico de madurez: ejercicio

Lee cada escenario y decide en qué nivel está. Argumenta tu respuesta.

Escenario A

Una startup de logística entrena cada semana un modelo de previsión de demanda. El data scientist tiene un notebook, lo ejecuta los lunes, exporta a `.pkl`, lo sube por SFTP al servidor de la API. La API recarga el modelo al reiniciarse. Hay un README de cinco líneas. No hay tests.

Escenario B

Un banco tiene un modelo de scoring de crédito. Cada commit a `main` dispara un workflow de GitHub Actions que entrena, evalúa contra un dataset golden, registra en MLflow y, si supera al actual, lo promociona a staging para QA manual. Producción se aprueba por PR.

Escenario C

Un retailer entrena un modelo de recomendación cada noche con Airflow, tiene un model registry, monitoring con Evidently, retraining automático si el drift supera un umbral, despliegue canary del 5%, y un dashboard que su SRE mira a las 9 de la mañana. Si algo falla, rollback automático.

8. Vocabulario mínimo

Término	Significado
Reproducibilidad	Capacidad de obtener exactamente el mismo modelo dadas las mismas entradas
Experiment	Conjunto de ejecuciones de entrenamiento para un mismo problema
Run	Una ejecución concreta dentro de un experiment
Model Registry	Almacén versionado de modelos con stages (Staging, Production, Archived)
Pipeline	Grafo de pasos automatizado (DVC, Airflow, Prefect, Dagster)
Drift	Cambio en la distribución de inputs (data drift) o de la relación input-output (concept drift)
Feature Store	Almacén compartido de features para train y serve
Model Card	Documento estandarizado que describe propósito, datos y limitaciones de un modelo
Audit trail	Registro completo y trazable de qué predicciones hizo cada modelo, cuándo y con qué inputs

9. Para profundizar

- Sculley et al. *Hidden Technical Debt in Machine Learning Systems* (NeurIPS 2015).
- Studer et al. *Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology* (2021).
- Google Cloud. *MLOps: Continuous delivery and automation pipelines in machine learning* (whitepaper).
- ml-ops.org. Hoja de ruta de la comunidad MLOps.
- Made With ML. <https://madewithml.com>

1. DataOps en cuatro principios

DataOps toma prestada la filosofía de DevOps y la aplica a los flujos de datos. Cuatro principios resumen el manifiesto:

1. **Orquestación.** Un pipeline declarativo. No scripts sueltos ejecutándose en máquinas distintas.
2. **Observabilidad.** En cada momento sabes en qué paso va, cuánto ha tardado y qué ha producido.
3. **Automatización.** Si tienes que ejecutar algo a mano, antes o después se te olvidará.
4. **Versionado.** Datos, código y parámetros. Los tres deben poder volverse atrás juntos.

2. Versionado de datos: por qué Git no basta

Git fue diseñado para texto. Trata cada binario como un blob completo y los diffs no significan nada para un CSV. Si commiteas un dataset de 500 MB:

- Cada cambio duplica el peso del repo.
- `git clone` se vuelve eterno.
- GitHub te bloquea por encima de 100 MB por fichero.
- El histórico crece de forma incontrolada.

Soluciones específicas para datos:

Herramienta	Idea	Cuándo usarla
DVC	Puntero en git, blob en remoto (S3, GCS, MinIO, Azure)	Equipos pequeños y medianos, integración con Git
LakeFS	Git para data lakes	Cuando ya trabajas con S3/GCS a gran escala
Delta Lake	Tablas con time travel sobre Spark/ Databricks	Stack Databricks o Spark
Iceberg	Formato de tabla con snapshots	Data lakes modernos, OLAP

En este curso usamos **DVC** porque encaja con Git, funciona en local con MinIO y enseña el patrón general muy bien.

3. Cómo funciona DVC

Cuando ejecutas `dvc add adult.csv`, DVC hace tres cosas:

1. Calcula el hash MD5 del fichero.
2. Mueve el fichero real a un almacén externo (S3, MinIO, etc.).
3. Crea un fichero pequeño `adult.csv.dvc` con la referencia al hash.

Git solo versiona el `.dvc` (pocos bytes). El blob real vive en el remoto. Para recuperarlo en otra máquina:

```
git clone repo
cd repo
dvc pull
```

DVC añade además una **caché por dependencias**: si en un pipeline una stage no ha cambiado, se salta. Como Make pero pensado para flujos de datos.

4. Calidad de datos: cinco dimensiones

Cualquier proyecto serio debería medir al menos estas cinco dimensiones de calidad:

Dimensión	Ejemplo
Compleitud	¿Cuántos nulos hay en campos obligatorios?
Validez	¿Los valores están dentro del rango o dominio esperado?
Unicidad	¿Hay duplicados que no deberían existir?
Consistencia	¿Cuadran los datos entre fuentes?
Freshness	¿El dato es de cuándo dice ser?

Herramientas para implementarlas:

- **Great Expectations.** El más maduro. Curva de aprendizaje notable.
- **Soda.** SQL-first, muy bueno para data engineers.
- **Pandera.** Ligero, integrado con pandas/polars.

En el lab usamos un GE simplificado en pandas puro. La razón es práctica: la API real de Great Expectations cambia entre versiones menores y no queremos perder tiempo de aula en eso. La lógica que implementaremos es exactamente la misma.

5. Orquestación: panorama

Herramienta	Modelo	Cuándo
Airflow	DAGs Python, scheduler central	Estándar de facto, pesado pero potente
Prefect	Funciones decoradas, dynamic DAGs	Buen punto intermedio, deploys sencillos
Dagster	Asset-based, software-defined assets	Cuando piensas en outputs, no en tareas

En el lab usamos **DVC pipelines** porque encajan con el ejercicio de versionado y son ligeros. En un proyecto real, casi siempre tendrás Airflow o Prefect orquestando por encima.

6. Lineage y catálogo

- **Lineage.** “Este dato vino de aquí, lo procesó tal script con tal versión, en tal momento”. OpenLineage es el estándar emergente.
- **Catálogo.** Dónde está cada dataset, quién es el owner, qué significa cada columna. DataHub (LinkedIn) y Amundsen (Lyft) son los más conocidos.

Para empresas con muchos datasets y muchos consumidores, son tan importantes como el versionado.

7. Lab 1. Pipeline reproducible con DVC y validación

Tiempo estimado: 50 minutos.

Objetivo

Montar un pipeline que valide y preprocese el dataset UCI Adult, con las siguientes garantías:

- El dataset no se commitea en Git.
- El blob real vive en MinIO via DVC.
- Las validaciones son código ejecutable.
- Un solo comando (`dvc repro`) reconstruye todo desde cero.

Prerrequisitos

```
docker compose -f docker/docker-compose.yml up -d minio postgres
cd labs/lab1_dataops
python -m venv .venv && source .venv/bin/activate
pip install dvc[s3]==3.55.2 pandas pyarrow scikit-learn pyyaml
```

Dataset

Utilizamos **UCI Adult Income** (~3.9 MB). Si no lo tienes:

```
mkdir -p data/raw
curl -L https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data \
-o data/raw/adult.csv
```

Si la red del aula falla, el dataset está en `datasets/adult/adult.csv` del propio repo del curso. Cópialo a `data/raw/adult.csv`.

Paso 1. Inicializar Git y DVC

```
git init -q
git add -A && git commit -q -m "snapshot inicial"
dvc init
git add .dvc/config .dvc/.gitignore .dvcignore
git commit -q -m "inicializa dvc"
```


Paso 2. Configurar el remoto MinIO

```
dvc remote add -d minio s3://datasets/lab1
dvc remote modify minio endpointurl http://localhost:9000
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345
git add .dvc/config && git commit -q -m "remoto minio configurado"
```

Observa la diferencia entre `dvc remote modify` (lo commitea) y `dvc remote modify --local` (lo guarda en `.dvc/config.local`, fuera de Git). Las credenciales **siempre** van en local.

Paso 3. Trackear el dataset

```
dvc add data/raw/adult.csv
cat data/raw/adult.csv.dvc          # ver el hash
cat data/raw/.gitignore             # ver que el csv real está excluido
git add data/raw/adult.csv.dvc data/raw/.gitignore
git commit -q -m "trackea dataset con dvc"
dvc push
```

Abre la consola de MinIO en `http://localhost:9001` (usuario `minio`, contraseña `minio12345`) y entra en el bucket `datasets`. Verás una carpeta con un fichero nombrado por el hash MD5 del CSV. Eso es DVC en su mínima expresión.

Paso 4. La suite de validación

El fichero `src/ge_validate.py` ya está escrito. Implementa cuatro validaciones:

- Schema (columnas esperadas presentes).
- Rango (age entre 17 y 90).
- Dominio (workclass dentro de un conjunto cerrado).
- Nulos (income no puede ser null).

Ejecuta:

```
python src/ge_validate.py data/raw/adult.csv
```

Salida esperada:

```
[GE] schema ok
[GE] age in [17, 90] ok
[GE] workclass in domain ok
[GE] no nulls on income ok
ALL EXPECTATIONS PASSED
```

Paso 5. El pipeline DVC

El fichero `dvc.yaml` declara dos stages:

```
stages:
  validate:
    cmd: python src/ge_validate.py data/raw/adult.csv
    deps:
      - data/raw/adult.csv
      - src/ge_validate.py
  preprocess:
    cmd: python src/preprocess.py
    deps:
      - data/raw/adult.csv
      - src/preprocess.py
      - params.yaml
    outs:
      - data/processed/train.parquet
      - data/processed/test.parquet
```

Ejecuta el pipeline:

```
dvc repro
```

Reejecuta:

```
dvc repro
```

Verás que DVC dice `didn't change, skipping` para ambas stages. Esto es la caché por dependencias.

Paso 6. Romper el dato a propósito

Vamos a comprobar que el pipeline para si los datos se corrompen.

```
sed -i '5s/.*/16, ?, 77516, Bachelors, 13, Never-married, ?, Not-in-family, White, Male, 2174, 0, 40, United-States, <=50K/' data/raw/adult.csv
dvc repro
```

Debe fallar con `VALIDATION FAILED: 1 rows with age outside [17, 90]`.

Recupera la versión correcta del CSV desde DVC:

```
dvc checkout data/raw/adult.csv
dvc repro
```

Vuelve a pasar.

Paso 7. Cambiar un parámetro

Edita `params.yaml` y cambia `test_size` de 0.2 a 0.3. Lanza:

```
dvc repro
```

DVC reejecuta **solo** la stage `preprocess` (no `validate`), porque sabe qué dependencia cambió.

Vuelve a 0.2 y commitea:

```
dvc repro
git add params.yaml dvc.lock
git commit -q -m "restaura test_size 0.2"
```

Paso 8. Simular un compañero nuevo

Esto demuestra la potencia del flujo:

```
cd /tmp
git clone ~/Documentos/Anbam/labs/lab1_dataops nuevo_companero
cd nuevo_companero
ls data/raw/      # no hay csv, solo el .dvc
dvc pull          # ahora sí
ls -lh data/raw/  # CSV recuperado
dvc repro         # reconstruye train y test
```

Cualquier máquina con acceso al remoto reconstruye el dataset y los artefactos exactos.

Checklist de cierre

- ☐ `git log` muestra varios commits sin el CSV pesado dentro.
- ☐ El bucket MinIO `datasets/lab1/` contiene los blobs.
- ☐ `dvc status` está limpio.
- ☐ Existen `data/processed/train.parquet` y `test.parquet`.
- ☐ Entiendes la diferencia entre `dvc.yaml` (definición) y `dvc.lock` (estado tras la última ejecución).

Reto opcional

Añade un stage `eda` al pipeline que genere un informe HTML con `pandas-profiling` o `ydata-profiling`. Decláralo como output del pipeline y commitealo.

8. Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
<code>dvc push</code> falla con 403	Credenciales MinIO mal escritas	Revisa <code>.dvc/config.local</code>
MinIO no levanta	Puerto 9000 ocupado	<code>lsof -i :9000</code> , mata el proceso o cambia el puerto
<code>dvc repro</code> se queja del YAML	Indentación mal	YAML solo admite espacios, no tabuladores
<code>sed -i</code> no funciona	Estás en macOS o Windows	Usa <code>sed -i '' '...'</code> (Mac) o un editor a mano

9. Para profundizar

- DVC. <https://dvc.org>. Tutoriales y casos de uso.
- Great Expectations. <https://greatexpectations.io>. Documentación oficial.
- Pandera. <https://pandera.readthedocs.io>. Validación lightweight.
- LakeFS. <https://lakefs.io>. Si en algún momento subes a escala.

Lab 1 — DataOps con DVC

Tiempo estimado: 40-50 minutos.

Qué vas a aprender

A versionar datasets con DVC y a montar un pipeline reproducible que valide y preprocese los datos. Al acabar tendrás:

- Un repositorio git con un dataset trackeado FUERA de git.
- Un pipeline declarativo en `dvc.yaml`.
- Train y test en parquet, generados por el pipeline.
- El dataset replicado en MinIO.

Cómo ejecutar el lab

Hay dos formas. La primera es la **recomendada en clase**.

Opción A — Modo asistido (recomendado)

Un solo comando. El script te va explicando lo que hace.

```
./run.sh
```

Tienes que pulsar Enter entre paso y paso. Lee lo que sale en pantalla, está pensado para que entiendas qué pasa.

Opción B — Modo manual

Sigue los pasos del PDF del módulo 2 tecleando los comandos tú mismo. Es lo que harías en tu trabajo real.

Antes de empezar

Tiene que estar levantado el stack:

```
# desde la raíz del repo:  
./setup.sh
```

Comprueba abriendo `http://localhost:9001` que ves la consola de MinIO.

Si algo falla

Ejecuta desde la raíz del repo:

```
./doctor.sh
```

Y comparte la salida con el profesor.

Estructura de este lab

```
lab1_dataops/
├── README.md           ← esto que lees
├── run.sh              ← script asistido (Opción A)
├── src/
│   ├── ge_validate.py  ← validaciones del dataset
│   └── preprocess.py   ← split en train/test
├── params.yaml         ← parámetros del preprocesado
└── dvc.yaml            ← se genera durante el lab
```

La carpeta `data/` se crea durante la ejecución. Los CSV grandes NUNCA viven dentro de Git, viven en MinIO via DVC.

Tras completar el lab

Comprobaciones que debes poder marcar:

- ☐ `git log --oneline` muestra varios commits y ningún CSV pesado dentro.
- ☐ `dvc status` dice que todo está limpio.
- ☐ Existen `data/processed/train.parquet` y `data/processed/test.parquet`.
- ☐ En MinIO (`http://localhost:9001` → bucket `datasets`) hay un blob con el hash del CSV.
- ☐ Entiendes qué pasó cuando ejecutamos `dvc repro` con el dato roto.

Cuando lo tengas, salta al lab siguiente:

```
cd ../lab2_mlflow_dvc && ./run.sh
```

1. El problema que resuelve el tracking

Sin un sistema de tracking, una sesión típica de modelado se parece a esto:

Ejecutas el notebook con `lr=0.1`. Sale F1 0.72. Cambias a 0.05. Sale 0.74. Cambias `max_depth` a 10. Sale 0.71. Vuelves a 0.05 y `max_depth` 5. Sale 0.76. Lo apuntas en un post-it. A las dos horas, ya no sabes qué configuración exacta dio el 0.76, ni qué dataset tenías, ni qué seed pusiste.

Multiplica esto por un equipo de cinco personas durante seis meses y tienes el escenario habitual de cualquier proyecto de ML sin disciplina.

Un sistema de tracking responde a tres preguntas básicas:

1. **Reproducibilidad.** Si guardas params + datos + seed + entorno, puedes recrear el modelo exacto.
2. **Comparación.** Cuando tienes 30 experimentos, ¿cuál era el bueno y por qué?
3. **Auditoría.** En sectores regulados te van a pedir el log completo.

2. Anatomía de MLflow

MLflow tiene cuatro componentes:

+-----+	+-----+
Tracking	Models
(params,	(formato y
metrics,	flavors)
artifacts)	
+-----+	+-----+
+-----+	+-----+
Projects	Model Registry
(reproducible	(versionado,
runs)	stages)
+-----+	+-----+

En este curso usamos **Tracking**, **Models** y **Registry**. Projects es opcional y se usa menos.

3. Vocabulario clave

Término	Significado
Experiment	Carpeta lógica. Un experiment representa un problema (ej. <code>income-classifier</code>)
Run	Una ejecución concreta de entrenamiento dentro del experiment
Parameter	Valor fijado antes del entrenamiento (hiperparámetro o config)
Metric	Valor medido. Puede ser un número final o una serie (por epoch)
Tag	Etiqueta libre clave/valor (ej. <code>git_commit=abc123</code>)
Artifact	Fichero asociado al run (modelo, gráficas, datasets, plots)
Signature	Schema de inputs/outputs del modelo
Model Registry	Capa por encima donde un modelo tiene nombre, versión y stage

Los **stages** de un modelo registrado son: `None` , `Staging` , `Production` , `Archived` . Es una convención: significan lo que tú quieras hacer con ellos, pero hay un consenso amplio.

4. Alternativas a MLflow

Herramienta	Punto fuerte	Punto débil
MLflow	Open source, autohospedable, estándar de facto	UI mejorable
Weights & Biases	UI excelente, colaboración fluida	SaaS, on-prem es de pago
Neptune	Bueno para deep learning	Menor comunidad
Comet	Muy completo	Comercial
Aim	Open source, rápido	Menos features que MLflow

En empresas con restricciones de datos (banca, salud, RGPD) suele ganar **MLflow**. En equipos de re-search donde la pasta no es problema, **Weights & Biases**.

5. Reproducibilidad estricta

Para que un experimento sea **realmente** reproducible necesitas:

1. **Código** (commit hash en Git).
2. **Datos** (hash del dataset, en nuestro caso vía DVC).
3. **Parámetros** (logueados en MLflow).
4. **Entorno** (requirements.txt + versión de Python, idealmente Docker).
5. **Seed** aleatoria fijada en cada librería estocástica.

Si te falta uno solo, no es reproducible. En la práctica, los seeds suelen estar en `random_state` para sklearn, `seed` para xgboost, `torch.manual_seed` para PyTorch, etc.

6. Lab 2. MLflow Tracking y Model Registry

Tiempo estimado: 50 minutos.

Objetivo

Entrenar tres clasificadores comparables, trackearlos en MLflow, registrar el mejor en el Model Registry y promocionarlo a `Staging`.

Prerrequisitos

- Lab 1 completo. Los ficheros `data/processed/train.parquet` y `test.parquet` deben existir.
- Stack levantado: `docker compose up -d`.

Variables de entorno

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
```

Para no teclear esto en cada terminal, guárdalo en un `setenv.sh` y lánzalo con `source setenv.sh`.

Paso 1. Repasar el código de entrenamiento

Abre `src/train.py`. Las partes clave son:

```
mlflow.set_tracking_uri(...)
mlflow.set_experiment("income-classifier")

with mlflow.start_run(run_name=args.model):
    mlflow.log_params(params)
    mlflow.set_tags({
        "git_commit": git_commit(),
        "dataset_version": "lab1-v1",
        "owner": args.owner,
        "framework": type(model).__name__,
    })
    model.fit(X_tr, y_tr)
    metrics = {...}
    mlflow.log_metrics(metrics)
    sig = mlflow.models.infer_signature(X_tr, y_pred)
    mlflow.sklearn.log_model(model, name="model", signature=sig,
                             input_example=X_tr.head(3))
```

Tres ideas:

- El `with mlflow.start_run(...)` cierra el run incluso si hay excepción.
- `log_params` y `log_metrics` aceptan diccionarios.
- `signature` es un esquema de inputs y outputs. Sin él, MLflow loguea pero te dará problemas al servir.

Paso 2. Entrenar tres modelos

```
cd labs/lab2_mlflow_dvc
python src/train.py --model logreg
python src/train.py --model rf
python src/train.py --model xgb
```

Cada ejecución crea un run nuevo en el experimento `income-classifier`. Por consola verás algo como:

```
== logreg == accuracy=0.8423 f1=0.6512 precision=0.7321 recall=0.5876
roc_auc=0.9012
```

Paso 3. Comparar runs en la UI

Abre `http://localhost:5000` en el navegador. Entra en el experimento `income-classifier`. Verás los tres runs.

- Marca los tres checkboxes.
- Pulsa **Compare**.
- Examina el gráfico de coordenadas paralelas.
- Ordena por `f1`. El ganador suele ser `xgb`.

Explora también la tab **Artifacts** de un run: verás el modelo serializado, su `MLmodel` (metadatos), el `environment` (`conda.yaml`, `python_env.yaml`) y el `input_example.json`.

Paso 4. Registrar el modelo ganador

Desde la UI:

1. Click en el run ganador.
2. Tab **Artifacts**.
3. Selecciona la carpeta `model`.
4. Botón **Register Model**.
5. Nombre: `income-clf`.
6. Confirma.

Recarga. Ahora tienes el modelo `income-clf` versión 1, stage `None`.

Por CLI hace lo mismo:

```
python src/register_best.py \
    --experiment income-classifier --metric f1 --name income-clf
```

Paso 5. Promocionar a Staging

```
python src/promote.py --name income-clf --version 1 --stage Staging
```

Recarga la UI. El modelo está en `Staging`.

Paso 6. Cargar desde el Registry

En un Python interactivo:

```
import mlflow.pyfunc
import os
os.environ["MLFLOW_TRACKING_URI"] = "http://localhost:5000"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://localhost:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

m = mlflow.pyfunc.load_model("models:/income-clf/Staging")
print(m.metadata.signature)
print(m.metadata.run_id)
```

Verás la signature completa con las columnas esperadas, y el `run_id` del entrenamiento. Esta línea (`models:/income-clf/Staging`) es la que usará la API del lab 3.

Checklist de cierre

- ☐ Hay 3 runs en el experimento `income-classifier`.
- ☐ Cada run tiene tags `git_commit`, `dataset_version`, `owner`, `framework`.
- ☐ Cada run tiene un modelo logueado con signature.
- ☐ Existe el modelo `income-clf` versión 1 en el Registry, en stage `Staging`.
- ☐ Puedes cargar el modelo desde Python con `models:/income-clf/Staging`.

Reto opcional

Añade un cuarto modelo usando `optuna` para búsqueda de hiperparámetros (20 trials). Compara con los tres anteriores y promociónalo si bate al ganador actual.

7. Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
<code>RESOURCE_DOES_NOT_EXIST</code> al lanzar <code>train.py</code>	Versión muy vieja de MLflow o permisos	Verifica con <code>mlflow experiments search</code>
<code>PermissionError</code> al loguear el modelo	Las credenciales MinIO no están exportadas	Reejecuta <code>source setenv.sh</code>
XGBoost no instala en Mac M1	Wheel no disponible	<code>pip install xgboost --no-binary :all:</code> o salta <code>xgb</code>
El Registry no aparece en la UI	MLflow muy antiguo	Actualiza a <code>>= 2.x</code>
<code>load_model</code> da avisos de versión	Diferencia menor de dependencias	Suele ser inofensivo. Si peta, usa <code>env_manager='conda'</code>

8. Buenas prácticas que pagan

1. **Usa siempre signature.** Es la diferencia entre un modelo y un pickle.
2. **Tagea todo.** `git_commit`, `dataset_version`, `owner`. Tu yo del futuro te lo agradecerá.
3. **Un experiment por problema.** No mezcles distintos casos en uno.
4. **Stage transitions explícitas.** Documenta quién promociona y por qué.

9. Para profundizar

- MLflow Documentation. <https://mlflow.org/docs/latest/index.html>
- *Designing Machine Learning Systems*, Chip Huyen. Capítulo de experimentos.
- Comparativa W&B vs MLflow: <https://wandb.ai/site/articles/mlflow-vs-wandb>

Lab 2 — MLflow Tracking + Model Registry

Tiempo estimado: 50 min

Objetivo

Trackear 3 modelos comparables, registrar el mejor en MLflow Model Registry y promocionarlo a `Staging` desde la API.

Pre-requisitos

- Lab 1 completo (`data/processed/{train,test}.parquet` existen).
- Stack `docker compose up -d` levantado (`mlflow` , `minio` , `postgres`).

Variables de entorno

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
```

Paso 1 · Entrenar 3 modelos

```
python src/train.py --model logreg
python src/train.py --model rf
python src/train.py --model xgb
```

Cada ejecución crea un run dentro del experimento `income-classifier` .

Paso 2 · Comparar runs

Abre <http://localhost:5000>:

- En el experimento `income-classifier` , marca los 3 runs y pulsa **Compare**.
- Examina el gráfico de coordenadas paralelas.
- Ordena por `f1` y elige el ganador.

Paso 3 · Registrar el modelo ganador

Desde la UI de MLflow → run ganador → tab **Artifacts** → seleccionar carpeta `model` → botón **Register Model**:

- Name: `income-clf`
- Version: 1 (auto)

O por CLI:

```
python src/register_best.py --experiment income-classifier --metric f1 --name income-clf
```

Paso 4 · Promocionar a Staging

```
python src/promote.py --name income-clf --version 1 --stage Staging
```

Paso 5 · Cargar el modelo desde el Registry

```
import mlflow.pyfunc
m = mlflow.pyfunc.load_model("models:/income-clf/Staging")
print(m.metadata.signature)
```

Comprobaciones de cierre

- ☐ Hay 3 runs con tags `dataset_version`, `git_commit` y `owner`.
- ☐ Cada run tiene `model` con signature.
- ☐ Existe un modelo `income-clf` en el Registry.
- ☐ Su versión 1 está en stage `Staging`.
- ☐ Puedes recargarlo y predecir.

Reto opcional

Añade un cuarto modelo con `optuna` (búsqueda de hiperparámetros) y sustituye al ganador.

DÍA 2 — MLOps en producción

Duración: 4 h (240 min) · **Carga:** teoría 50 % · práctica 50 %

Hoja de ruta del día

Hora aprox.	Bloque	Tipo
00:00 – 00:35	Módulo 4 — Empaquetado y serving (FastAPI)	Teoría
00:35 – 01:15	Lab 3 — FastAPI + Docker	Práctica
01:15 – 02:05	Módulo 5 — CI/CD + Monitoring + Drift	Teoría
02:05 – 02:55	Lab 4 — GitHub Actions + Evidently	Práctica
02:55 – 03:05	Pausa	—
03:05 – 03:55	Módulo 6 — Caso integrador + Governance	Mixto
03:55 – 04:00	Cierre, Q&A y hoja de ruta	—

Prerrequisitos del Día 2

- Tienes el modelo del Día 2 promocionado en MLflow a `Staging` (Lab 2).
- El stack docker está arriba: `docker compose ps` muestra los cinco servicios `Up`.
- Si reiniciaste el equipo: `docker compose start` (no `up`) para conservar los runs.

Si te falta algo de lo anterior, repasa el Lab 2 antes de continuar — los labs del Día 2 dependen del modelo registrado.

Lo que debes lograr al final del Día 2

- Servir el modelo registrado como API REST con FastAPI dentro de un contenedor Docker.
- Tener un pipeline CI/CD que entrena, evalúa, promociona y construye la imagen automáticamente.
- Detectar *data drift* con Evidently y razonar qué falta para llevar el sistema a producción real.

1. Patrones de serving

No todos los modelos se sirven igual. Hay cuatro patrones clásicos:

Patrón	Cuándo se usa	Ejemplo
Batch	Predicciones masivas periódicas	Scoring nocturno de una cartera
Online	Respuesta en milisegundos por petición	API REST/gRPC en una app
Streaming	Datos llegando por una cola	Kafka + consumer ML
Edge	Sin red, en el propio dispositivo	Móviles, IoT, embedded

En este módulo vemos **online**, que es el más común y la base para muchos casos.

2. Cómo se empaqueta un modelo

De menos a más sofisticado:

1. **Pickle de sklearn.** Funciona, pero queda atado a la versión de Python y librerías. Si actualizas algo, deja de cargar.
2. **ONNX.** Formato neutral. Te lleva el modelo a cualquier lenguaje o runtime. Muy rápido en producción.
3. **MLflow flavor.** Lo que usamos en este curso. Empaqueta el modelo con todos los metadatos de su entorno.
4. **Servers especializados.** BentoML, TorchServe, TF Serving, Triton. Hacen batching dinámico, A/B testing nativo, etc.

3. Cuatro principios de diseño de la API

1. **Contrato explícito.** Define los inputs con Pydantic (FastAPI) o marshmallow (Flask). Si el cliente manda mal el payload, la API responde 422 antes de tocar el modelo.
2. **Endpoints separados.**
 - `/predict` para inferencia.
 - `/health` para load balancers y readiness probes.
 - `/version` para auditoría rápida (qué modelo está cargado).
 - `/metrics` para Prometheus.
3. **Versionado.** O bien con prefijo en la ruta (`/v1/predict` , `/v2/predict`), o bien con header de versión. Lo que no puedes hacer es romper el contrato sin avisar.
4. **Logging trazable.** Cada request con su `request_id` . Si un cliente reclama, puedes reconstruir qué se le pidió al modelo.

4. Métricas de rendimiento

Métrica	Qué significa
Latencia p50	La mitad de las peticiones tardan menos que esto
Latencia p95	El 95% tardan menos. Útil para SLAs
Latencia p99	El peor 1% queda por debajo. Importante en escala
Throughput (RPS)	Peticiones por segundo que aguanta
Cold start	Tiempo desde frío hasta la primera respuesta

No te quedes con la media: el p95 y p99 cuentan la historia real. Una API con media de 100 ms y p99 de 5 s no es aceptable en producción.

5. Trade-offs típicos

Dos discusiones que salen siempre:

1. **Latencia vs frescura del modelo.** Si recargas el modelo en caliente cada hora, tendrás glitches en el momento de la recarga. Si no recargas, sirves un modelo posiblemente obsoleto.
2. **Monolítico vs microservicios.** Un servicio por modelo es limpio pero multiplica infraestructura. Un servicio único con router (varios modelos en una sola API) es eficiente pero más acoplado.

6. Lab 3. API REST en contenedor

Tiempo estimado: 40 minutos.

Objetivo

Servir el modelo `income-clf@Staging` (registrado en el lab 2) como una API REST en contenedor, con endpoints documentados, validación de entrada y benchmarking básico.

Arquitectura

```
[ cliente ] → [ FastAPI :8000 ] → modelo cargado al iniciar
```

- `/predict` (POST)
- `/health` (GET)
- `/version` (GET)
- `/metrics` (GET, Prometheus)

Prerrequisitos

- El modelo `income-clf` versión 1 está en `Staging` (lab 2 completo).
- Stack levantado.
- Dependencias de FastAPI instaladas.

```
cd labs/lab3_serving
pip install -r requirements.txt
```

Paso 1. Arrancar en local

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
export MODEL_URI=models:/income-clf/Staging

uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

La primera vez tarda 10-20 segundos en cargar el modelo. Cuando veas `model loaded • run_id=...`, abre en el navegador:

- `http://localhost:8000/docs` → Swagger interactivo.
- `http://localhost:8000/redoc` → vista alternativa.

Paso 2. Probar con curl

Desde otra terminal:

```
curl -s http://localhost:8000/health | jq
```

```
curl -s http://localhost:8000/version | jq
```

Predicción:

```
curl -s -X POST http://localhost:8000/predict \
-H 'content-type: application/json' \
-d '{
  "age": 39,
  "workclass": "State-gov",
  "education_num": 13,
  "marital_status": "Never-married",
  "occupation": "Adm-clerical",
  "relationship": "Not-in-family",
  "race": "White",
  "sex": "Male",
  "capital_gain": 2174,
  "capital_loss": 0,
  "hours_per_week": 40,
  "native_country": "United-States"
}' | jq
```

Respuesta:

```
{
  "label": 0,
  "proba": 0.18,
  "model_uri": "models:/income-clf/Staging",
  "request_id": "a3f8c2d4e9b0",
  "latency_ms": 4.21
}
```

Prueba ahora con un payload inválido (por ejemplo `"age": 200`). Debe devolver **422 Unprocessable Entity** sin tocar el modelo. Eso es validación Pydantic.

Paso 3. Métricas en formato Prometheus

```
curl -s http://localhost:8000/metrics
```

Salida:

```
# HELP api_predictions_total total predictions served
# TYPE api_predictions_total counter
```

```
api_predictions_total 5
# HELP api_errors_total total errors
# TYPE api_errors_total counter
api_errors_total 0
# HELP api_latency_avg_ms average inference latency
# TYPE api_latency_avg_ms gauge
api_latency_avg_ms 4.83
```

Cualquier Prometheus puede hacer scraping de este endpoint.

Paso 4. Construir la imagen Docker

Para el lab usamos un Dockerfile multi-stage:

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
COPY app/ ./app/
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Compila:

```
docker build -t anban/income-api:0.1 -f Dockerfile .
docker images anban/income-api
```

La imagen final debería pesar entre 250 y 350 MB. Si pesa más, revisa qué dependencias estás incluyendo.

Paso 5. Lanzar el contenedor

En Linux (puedes usar la red del host):

```
docker run --rm -d --network host \
  -e MLFLOW_TRACKING_URI=http://localhost:5000 \
  -e MLFLOW_S3_ENDPOINT_URL=http://localhost:9000 \
  -e AWS_ACCESS_KEY_ID=minio \
  -e AWS_SECRET_ACCESS_KEY=minio12345 \
  -e MODEL_URI=models:/income-clf/Staging \
  --name income-api \
  anban/income-api:0.1

sleep 5
curl -s http://localhost:8000/health | jq
```

En macOS / Windows, conecta a la red de Docker Compose:

```
docker run --rm -d --network anban_default \
  -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MLFLOW_S3_ENDPOINT_URL=http://anban-minio:9000 \
  ...
```

Paso 6. Stress test con Locust

Locust simula usuarios concurrentes y mide latencias reales bajo carga.

Crea `tests/locustfile.py`:

```
from locust import HttpUser, task, between

PAYLOAD = {
    "age": 39, "workclass": "Private", "education_num": 13,
    "marital_status": "Never-married", "occupation": "Tech-support",
    "relationship": "Not-in-family", "race": "White", "sex": "Male",
    "capital_gain": 0, "capital_loss": 0,
    "hours_per_week": 40, "native_country": "United-States"
}

class APIUser(HttpUser):
    wait_time = between(0.1, 0.5)
    @task
    def predict(self):
        self.client.post("/predict", json=PAYLOAD)
```

Lanza:

```
pip install locust
locust -f tests/locustfile.py --host http://localhost:8000 \
  --users 50 --spawn-rate 10 --run-time 30s --headless
```

Criterios de aprobado típicos:

- p95 < 100 ms.
- 0 fallos.
- RPS sostenido > 200.

Checklist de cierre

- ☐ La API responde a `/health`, `/version`, `/predict`, `/metrics`.
- ☐ La imagen Docker pesa menos de 350 MB.
- ☐ El contenedor carga el modelo desde el Registry sin problema.

- ☐ Pydantic devuelve 422 ante entradas inválidas (age=200, por ejemplo).
- ☐ `/predict` devuelve label, proba, request_id y latencia.

Reto opcional

Añade un endpoint `/predict/batch` que acepte hasta 1000 registros y haga un único `model.predict` vectorizado. Mide cuánto mejora el RPS.

7. Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
<code>Connection refused</code> al cargar el modelo desde el contenedor	El contenedor no puede ver MLflow	En Mac/Win usa <code>host.docker.internal</code> . En Linux, <code>--network host</code>
<code>MlflowException: Could not find any artifacts</code>	Credenciales MinIO no exportadas en el contenedor	Pasa <code>AWS_ACCESS_KEY_ID</code> y <code>AWS_SECRET_ACCESS_KEY</code> con <code>-e</code>
<code>KeyError</code> al alinear columnas	Categoría nueva no vista en train	El código ya lo maneja con default 0; revisa que no rompiste <code>_to_frame</code>
Uvicorn reload se vuelve loco	El reload se reactiva en cada save	Quita <code>--reload</code> para entornos productivos

8. Para profundizar

- FastAPI. <https://fastapi.tiangolo.com>. Documentación excelente.
- BentoML. <https://docs.bentoml.org>. Cuando necesites un server más serio.
- *Building Machine Learning Powered Applications*, Emmanuel Ameisen. Capítulos de productivización.

Lab 3 — FastAPI + Docker para servir el modelo

Tiempo estimado: 40 min

Objetivo

Exponer el modelo `income-clf@Staging` como una API REST en contenedor con endpoints documentados, validación de input y benchmarking básico.

Arquitectura

```
[ cliente ] → [ FastAPI :8000 ] → carga modelo desde MLflow Registry
```

- /predict (POST)
- /health (GET)
- /version (GET)
- /metrics (GET, prometheus)

Ejecución local (sin Docker)

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
export MODEL_URI=models:/income-clf/Staging

uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Abre <http://localhost:8000/docs> → Swagger interactivo.

Probar la API

```
curl -s http://localhost:8000/health | jq

curl -s -X POST http://localhost:8000/predict \
  -H 'content-type: application/json' \
  -d '{
    "age": 39,
    "workclass": "State-gov",
    "education_num": 13,
    "marital_status": "Never-married",
    "occupation": "Adm-clerical",
    "relationship": "Not-in-family",
```

```
"race": "White",  
"sex": "Male",  
"capital_gain": 2174,  
"capital_loss": 0,  
"hours_per_week": 40,  
"native_country": "United-States"  
' | jq
```

Construir la imagen

```
docker build -t anban/income-api:0.1 -f Dockerfile .  
docker run --rm -p 8000:8000 \  
-e MLFLOW_TRACKING_URI=http://host.docker.internal:5000 \  
-e MODEL_URI=models:/income-clf/Staging \  
anban/income-api:0.1
```

En Linux puro reemplaza `host.docker.internal` por la IP del host o usa la red `host` de Docker.

Stress test con Locust

```
locust -f tests/locustfile.py --host http://localhost:8000 \  
--users 50 --spawn-rate 10 --run-time 1m --headless
```

Aprobamos si: - p95 latency < 100ms - 0 fallos - RPS sostenido > 200

Comprobaciones de cierre

- ☐ `/health` responde en < 50ms.
- ☐ `/version` devuelve `model_uri`, `run_id` y `signature`.
- ☐ `/predict` valida input (Pydantic) y devuelve `label` + `proba`.
- ☐ Imagen final < 350MB (`docker images`).
- ☐ Locust cumple SLA.

Reto opcional

Añade `/predict/batch` que acepte una lista de hasta 1000 registros y hace un único `model.predict_batch`.

1. CI, CD y CT

Tres siglas que suelen confundirse:

- **CI** (Continuous Integration). Cada commit pasa por test, lint y build. Garantiza que el repo no se rompe.
- **CD** (Continuous Delivery o Continuous Deployment). La salida del CI se puede desplegar a producción con un clic (delivery) o automáticamente (deployment).
- **CT** (Continuous Training). Específico de ML. Reentrenamiento automático cuando llegan datos nuevos o cuando se detecta drift.

El triple bucle ML

```
commit → CI (test, lint, train, eval) → registry
                                     ↓
nuevos datos → CT (reentrena) → eval vs prod
                                     ↓
schedule → CD (despliega si bate al actual)
```

El bucle de la izquierda es el de DevOps. Los otros dos son los que añade MLOps.

2. Un pipeline CI/CD típico en GitHub Actions

Los jobs que casi siempre aparecen en clientes:

1. **lint + test.** Ruff, mypy, pytest. Barato, rápido, atrapa el 80% de bugs.
2. **train.** Entrena el modelo. En CI con dataset reducido.
3. **evaluate.** Compara contra dataset golden. Si no bate, fail.
4. **register.** Si pasa, registra la versión en el Model Registry.
5. **build.** Construye la imagen Docker.
6. **scan.** Análisis de vulnerabilidades (Trivy, Snyk).
7. **deploy.** Aplica el cambio (kubect!, helm, terraform).

Tres reglas que aprender pronto:

- En CI **no entrenes el modelo grande.** Los runners gratuitos son cortos y caros. El entrenamiento serio va a Airflow, Kubeflow o cluster propio.
- Los **secrets** nunca van al repo. Usa GitHub Encrypted Secrets o Vault.
- Si tu pipeline tarda 40 min en cada commit, la gente lo saltará. Mantenlo barato.

3. Estrategias de despliegue

Estrategia	Mecanismo	Cuándo
Blue-green	Dos entornos paralelos; switch del tráfico	Crítico, rollback en segundos
Canary	5% → 25% → 100% del tráfico	Detectar bugs sin daño masivo
Shadow	El nuevo recibe tráfico real pero no responde	Comparar nuevo vs antiguo sin riesgo

En ML, **shadow** es valiosísimo: te permite ejecutar el modelo nuevo en paralelo durante una semana con tráfico real antes de sustituir nada.

4. Monitoring: dos familias de métricas

Métricas de servicio (las de cualquier API):

- Latencia (p50, p95, p99).
- Throughput (RPS).
- Tasa de errores.
- Uso de CPU y memoria.

Métricas de modelo (específicas de ML):

- Distribución de outputs. ¿El modelo ahora dice “sí” un 80% en vez del 20% habitual?
- Accuracy proxy. Cuando tardas en tener el ground truth (ej. morosidad a 90 días), usas un proxy.
- Drift de inputs (lo siguiente).

5. Drift: tres sabores

Tipo	Qué cambia	Ejemplo
Data drift	$P(X)$	La distribución de la edad de los clientes se desplaza
Concept drift	$P(y X)$	Lo que antes predecía morosidad ya no lo predice
Label drift	$P(y)$	Antes 20% morosos; ahora 35%

Tests estadísticos habituales:

Test	Aplicación	Comentario
PSI	Numéricas	Umbrales intuitivos: 0.1 ok, 0.1-0.25 watch, > 0.25 drift
KS	Numéricas	Compara distribuciones, devuelve un p-value
Chi ²	Católicas	Mide independencia entre dos católicas
Wasserstein	Numéricas	Distancia "earth mover"

Population Stability Index (PSI)

PSI es el más usado en industria por su simplicidad. Para una variable numérica:

1. Divide la variable en N bins (típicamente 10 deciles del dataset de referencia).
2. Calcula la proporción de registros que caen en cada bin para la referencia (`expected_i`) y para el dataset actual (`actual_i`).
3. Suma sobre todos los bins:

$$PSI = \sum_i (actual_i - expected_i) * \ln(actual_i / expected_i)$$

Interpretación operativa:

- $PSI < 0.1$: distribución estable, sin acción.
- $0.1 \leq PSI < 0.25$: cambio moderado, vigilar.
- $PSI \geq 0.25$: drift significativo, investigar y posiblemente reentrenar.

6. Herramientas de drift

Herramienta	Tipo	Cuándo
Evidently	Open source, informes HTML	El estándar abierto, lo que vamos a usar
WhyLabs	SaaS, observabilidad full	Producción a escala con presupuesto
Arize	SaaS, especializado en NLP	Modelos de lenguaje en producción
NannyML	Open source	Estima accuracy sin labels

7. Lab 4. CI/CD con GitHub Actions y drift con Evidently

Tiempo estimado: 50 minutos.

Objetivo

1. Revisar un workflow de GitHub Actions que entrena, evalúa y promociona modelos según mejora.
2. Generar un reporte de drift con Evidently y exponerlo en la API.

Estructura

```
labs/lab4_cicd_monitoring/
├── .github/workflows/ml.yml      # workflow CI/CD
├── src/promote_if_better.py      # promoción condicional
├── src/drift_report.py           # genera HTML + JSON con Evidently
├── synthetic/make_drift.py       # crea un dataset con drift inyectado
└── extra/monitor_endpoint.py     # parche del /monitor para la API
```

Paso 1. Repasar el workflow

Abre `.github/workflows/ml.yml`. Lo importante:

```
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
```

Triggers en push a main, PRs y a mano (botón “Run workflow”).

```
jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest -q
```

Job barato (1-2 min) que se ejecuta en cada push.

```
train-eval:
  needs: lint-test
  ...
```

Encadenado con `needs: lint-test`. Si lint o tests fallan, no se entrena.

```
build-image:
  needs: train-eval
  if: github.ref == 'refs/heads/main'
```

Solo construye imagen en main, no en PRs.

Paso 2. Promoción condicional en local

Abre `src/promote_if_better.py`. La lógica clave:

```
candidate_metric = candidate_run.data.metrics[args.metric]
prod_metric = get_production_metric(...)
if prod_metric is None or candidate_metric >= prod_metric * (1 +
args.min_improvement):
    client.transition_model_version_stage(
        name=args.name,
        version=args.candidate_version,
        stage="Production"
    )
    print("PROMOTED"); sys.exit(0)
else:
    print("REJECTED"); sys.exit(1)
```

El parámetro `--min-improvement 0.01` significa “solo promociona si mejora al menos un 1%”. Esto evita oscilaciones.

Para probarlo, primero pon un modelo en Production:

```
python ../lab2_mlflow_dvc/src/promote.py \
  --name income-clf --version 1 --stage Production
```

Entrena otro y registra como v2:

```
python ../lab2_mlflow_dvc/src/train.py --model logreg
python ../lab2_mlflow_dvc/src/register_best.py \
  --experiment income-classifier --metric f1 --name income-clf
```

Prueba el promote condicional:

```
python src/promote_if_better.py \
  --name income-clf \
  --candidate-version 2 \
```

```
--metric f1 \  
--min-improvement 0.01
```

Normalmente sale **REJECTED** (logreg no bate a rf). Es lo esperado.

Ahora entrena un xgb (que sí suele batir):

```
python ../lab2_mlflow_dvc/src/train.py --model xgb  
python ../lab2_mlflow_dvc/src/register_best.py \  
    --experiment income-classifier --metric f1 --name income-clf  
python src/promote_if_better.py \  
    --name income-clf --candidate-version 3 \  
    --metric f1 --min-improvement 0.01
```

Debe salir **PROMOTED**.

Paso 3. Generar un dataset con drift

```
python synthetic/make_drift.py  
ls data/processed/      # aparece drifted.parquet
```

El script simula tres tipos de drift:

- Shift numérico en `age` (la población se rejuvenece).
- Concentración en cero de `capital_gain`.
- Cambio de proporciones en `marital_status`.

Paso 4. Generar el informe de drift

```
mkdir -p reports  
python src/drift_report.py \  
    --reference ../lab1_dataops/data/processed/test.parquet \  
    --current data/processed/drifted.parquet \  
    --output reports/drift.html
```

Abre `reports/drift.html` en el navegador. Si Evidently está disponible, verás un informe completo con distribuciones, métricas globales y mapa de correlaciones. Si no, el script cae a un fallback con PSI puro.

Recórrelo y observa:

- Métricas globales: ¿cuántas features driftearon?
- Detalle por feature: ¿en cuáles es más severo?
- Correlaciones: ¿alguna relación cambió de signo?

Paso 5. Endpoint /monitor en la API

El fichero `extra/monitor_endpoint.py` contiene un patch para añadir un endpoint `/monitor` a la API del lab 3:

```
@app.get("/monitor")
def monitor() -> dict[str, Any]:
    ref = pd.read_parquet(REF_PATH)
    cur = pd.read_parquet(CUR_PATH)
    scores = {col: psi(ref[col], cur[col])
              for col in ref.columns if col in cur.columns}
    drifted = [c for c, v in scores.items() if v > 0.25]
    return {
        "drift_score_overall": float(np.mean(list(scores.values()))),
        "drifted_features": drifted,
        "computed_at": datetime.utcnow().isoformat() + "Z",
    }
```

Copia este código al `main.py` del lab 3 (o aplica el patch como prefieras). Reinicia la API:

```
cd ../lab3_serving
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Y prueba:

```
curl -s http://localhost:8000/monitor | jq
```

Respuesta:

```
{
  "drift_score_overall": 0.34,
  "drifted_features": ["age", "capital_gain"],
  "computed_at": "2026-05-15T10:23:45Z"
}
```

Checklist de cierre

- ☐ Has leído y entendido el `ml.yml` (no necesitas ejecutarlo en GitHub).
- ☐ `promote_if_better.py` ha rechazado un modelo peor y aceptado uno mejor.
- ☐ `drift.html` se ha generado con datos esperados.
- ☐ El endpoint `/monitor` responde con `drift_score_overall`.

Reto opcional

Configura un trigger `schedule` en el workflow (cron semanal) que ejecute `make_drift.py` + `drift_report.py` y, si `drift_score_overall > 0.25`, dispare un retraining automático.

8. Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
Evidently no instala	Conflictos de dependencias	El script cae a fallback PSI, no pasa nada
<code>promote_if_better</code> falla	Aún no hay Production	El script lo maneja con <code>None</code>
Workflow YAML mal indentado	Tabuladores en vez de espacios	Solo espacios. Valida con <code>act</code> o <code>yamllint</code>
Drift report muy lento	Datasets demasiado grandes	Muestrea, no necesitas el dataset completo para drift

9. Para profundizar

- Evidently. <https://docs.evidentlyai.com>. Documentación con ejemplos.
- *Reliable Machine Learning*, Cathy Chen et al. (O'Reilly). Capítulos de monitoring.
- Charla “Detecting drift in production” — MLOps Community talks.

Lab 4 — CI/CD con GitHub Actions + Monitoring con Evidently

Tiempo estimado: 50 min

Objetivo

1. Pipeline GitHub Actions que entrene, evalúe y promocione el modelo si bate al de Production.
2. Reporte Evidently comparando dataset de referencia vs uno con drift.
3. Endpoint `/monitor` en la API que devuelva métricas de drift.

Estructura

```
.github/workflows/ml.yml    # workflow principal
src/promote_if_better.py    # promueve solo si supera al actual
src/drift_report.py         # genera HTML + JSON con Evidently
synthetic/make_drift.py     # crea un dataset 'drifted'
```

Paso 1 · Workflow GitHub Actions

Ya está en `.github/workflows/ml.yml`. Stages:

1. **lint+test** — `ruff + pytest`
2. **train** — `python src/train.py --model rf` con MLflow apuntando a un tracking server temporal (en CI usamos un tracking local en SQLite + artifacts en `./mlruns` que subimos como artifact del job)
3. **evaluate-and-promote** — compara F1 con la versión Production y transiciona si
`f1_new ≥ f1_prod * 1.01`

Para que funcione el job necesitas estos secrets en el repo:

- `MLFLOW_TRACKING_URI` (o usa el local en CI)
- `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` (si MinIO/S3 remoto)

Paso 2 · Probar el promote local

```
python src/promote_if_better.py \
  --name income-clf \
  --candidate-version 2 \
  --metric f1 \
  --min-improvement 0.01
```


Sale 0 si promociona, 1 si rechaza.

Paso 3 · Generar drift sintético y reporte

```
python synthetic/make_drift.py          # crea data/processed/drifted.parquet
python src/drift_report.py \
  --reference ../lab1_dataops/data/processed/test.parquet \
  --current data/processed/drifted.parquet \
  --output reports/drift.html
```

Abrir `reports/drift.html` en el navegador.

Paso 4 · Endpoint /monitor

Añade el endpoint del Lab 4 a la API del Lab 3 (parche en `extra/monitor_endpoint.py`). Reinicia, llama:

```
curl -s http://localhost:8000/monitor | jq
```

Devuelve:

```
{
  "drift_score_overall": 0.34,
  "drifted_features": ["hours_per_week", "capital_gain"],
  "computed_at": "2026-05-09T10:23:45Z"
}
```

Comprobaciones de cierre

- ☐ El workflow corre en push y muestra el resultado.
- ☐ `promote_if_better.py` rechaza un modelo peor.
- ☐ `drift.html` se ha generado correctamente.
- ☐ `/monitor` responde con `drift_score_overall`.

Reto opcional

Configura un **schedule** semanal en el workflow que haga reentrenamiento automático si el `drift_score_overall` > 0.25.

1. Feature Store: cuándo te hace falta

Imagina que tu modelo necesita la edad del cliente. Esa feature se calcula en dos sitios:

- **Train.** Notebook con SQL contra el data warehouse, corte temporal hace 6 meses.
- **Serve.** API de producción que llama a microservicio de perfil, corte temporal “ahora”.

¿Te aseguras de que ambas definiciones dan el mismo número para el mismo cliente? Casi nunca. Esto se llama **train/serve skew** y es una de las causas habituales de “el modelo funciona en mi notebook pero en producción no”.

Una **Feature Store** define cada feature una sola vez y la consumen ambos lados:

Feature Store	Tipo	Cuándo plantearlo
Feast	Open source	Cuando empiezas a duplicar transformaciones
Tecton	SaaS	Si tienes presupuesto y casos serios
Hopsworks	Open + commercial	Plataforma completa
Databricks Feature Store	Si ya estás en Databricks	Encaja muy bien con su stack

Cuándo NO la necesitas. Al principio, no la necesitas. Solo cuando llevas tres modelos productivos que comparten features y ya has visto el train/serve skew en directo.

2. Model Governance

Cinco bloques a cuidar:

2.1 Model Card

Documento estandarizado (propuesto por Mitchell et al., Google, 2019) que describe:

- Propósito del modelo.
- Datos usados (origen, periodo, sesgos conocidos).
- Métricas globales y por subgrupo.
- Limitaciones y contraindicaciones.
- Decisiones que NO debe automatizar.
- Mantenedor y owner.

Sin Model Card, dentro de un año nadie sabrá qué hacía ese modelo.

2.2 Audit trail

Registro completo de qué predicciones hizo cada modelo, cuándo, con qué inputs. Obligatorio en banca, salud y seguros. Útil en cualquier sector.

Mínimo viable:

- Log estructurado de cada request con `request_id`, `model_version`, `inputs` (resumidos), `output`, `timestamp`.
- Conservación según regulación local (en banca europea, mínimo 5 años).

2.3 RGPD y AI Act

RGPD Art. 22: si tu modelo toma decisiones automatizadas con efecto jurídico sobre personas (denegar crédito, contratar, asignar becas), el afectado tiene derecho a:

- Intervención humana.
- Expresar su punto de vista.
- Impugnar la decisión.

AI Act europeo (entrada en vigor escalonada hasta 2026): clasifica los sistemas IA por riesgo (mínimo, limitado, alto, inaceptable) e impone obligaciones específicas. Los modelos de “alto riesgo” (scoring crediticio, recursos humanos, justicia) tienen requisitos muy estrictos.

2.4 Sesgo y fairness

Una métrica global enmascara disparidades. Calcula siempre:

- F1, precision, recall **por subgrupo** (sex, race, edad).
- Tasa de falsos positivos por subgrupo.
- Disparidad de oportunidades igualadas.

Si encuentras disparidad grande, decisiones posibles:

- Reentrenar con técnicas de fairness (reweighing, adversarial debiasing).
- Cambiar el umbral por subgrupo.
- Documentarlo y aceptarlo (con justificación).
- No desplegar.

2.5 Decisión humana

Para casos críticos, **siempre** debe haber un humano en el bucle:

- Denegación de crédito.
- Diagnóstico médico.
- Selección de personal.
- Reconocimiento facial en espacios públicos.

El modelo recomienda; un humano decide.

3. Reproducibilidad estricta

Para reconstruir un modelo dentro de un año necesitas conservar:

Componente	Cómo se versiona
Código	git commit hash
Datos	DVC hash MD5
Parámetros	params.yaml o MLflow params
Entorno	requirements.txt + Python version + Docker image hash
Seed	random_state fijado en cada librería estocástica

Si tienes los cinco, eres reproducible. Si te falta uno, no.

4. Cost y FinOps en ML

Algunos órdenes de magnitud (aproximados, varían según cloud):

- Entrenamiento BERT base en GPU cloud: 50€ - 200€.
- Inferencia en CPU vs GPU: hasta **100x** diferencia de coste para modelos pequeños.
- Endpoint 24/7 dedicado vs serverless: hasta **10x** diferencia para cargas bajas.

Preguntas que tu equipo debería responder antes de elegir:

1. ¿Online o batch? Si toleras 1 hora de latencia, batch es 10x más barato.
2. ¿GPU o CPU? Para modelos pequeños, CPU sobra.
3. ¿Autoscaling? Si tu tráfico es 9-18h, no pagues 24/7.
4. ¿Spot instances? Hasta 70% de descuento para cargas tolerantes a interrupción.

5. LLMOps en un vistazo

Términos esenciales para entender la conversación actual:

- **Prompt versioning.** Cada prompt es código y se versiona.
- **Eval.** Medir la salida de un LLM es difícil. Herramientas: Promptfoo, LangSmith, Ragas (para RAG).
- **Guardrails.** Validar inputs y outputs. Bibliotecas como Guardrails AI o NeMo Guardrails.
- **RAG.** Retrieval-Augmented Generation. Conectas el LLM a tu base documental para responder con tus datos.
- **Embeddings y vector DB.** Pinecone, Weaviate, pgvector, Qdrant.

Si te interesa profundizar, hay un curso aparte específico.

6. Lab 5. Recorrido end-to-end y governance

Tiempo estimado: 45 minutos.

Objetivo

Recorrer el ciclo completo en una sola sesión:

- DataOps (lab 1).
- Tracking + Registry (lab 2).
- Serving (lab 3).
- Drift + promoción (lab 4).

Y rellenar la **Model Card** del modelo entrenado.

El recorrido en 8 comandos

```
cd ~/Documentos/Anbam

# 1. Stack y deps
docker compose -f docker/docker-compose.yml up -d
pip install -r requirements.txt

# 2. Datos
cd labs/lab1_dataops
dvc pull
dvc repro
cd ../..

# 3. Tests
pytest -q labs/

# 4. Entrenamiento con tracking
python labs/lab2_mlflow_dvc/src/train.py --model rf

# 5. Promoción
python labs/lab2_mlflow_dvc/src/register_best.py \
    --experiment income-classifier --metric f1 --name income-clf
python labs/lab2_mlflow_dvc/src/promote.py \
    --name income-clf --version 1 --stage Production

# 6. Servir
docker build -t anban/income-api:final -f labs/lab3_serving/Dockerfile labs/
lab3_serving
docker run -d --network host --name income-api \
    -e MLFLOW_TRACKING_URI=http://localhost:5000 \
    -e MLFLOW_S3_ENDPOINT_URL=http://localhost:9000 \
    -e AWS_ACCESS_KEY_ID=minio \
```



```

-e AWS_SECRET_ACCESS_KEY=minio12345 \
-e MODEL_URI=models:/income-clf/Production \
anban/income-api:final

sleep 8
curl -s http://localhost:8000/version | jq

# 7. Drift
python labs/lab4_cicd_monitoring/synthetic/make_drift.py
python labs/lab4_cicd_monitoring/src/drift_report.py \
    --reference labs/lab1_dataops/data/processed/test.parquet \
    --current labs/lab4_cicd_monitoring/data/processed/drifted.parquet \
    --output reports/drift_final.html

# 8. Promote conditional
python labs/lab2_mlflow_dvc/src/train.py --model xgb
python labs/lab2_mlflow_dvc/src/register_best.py \
    --experiment income-classifier --metric f1 --name income-clf
python labs/lab4_cicd_monitoring/src/promote_if_better.py \
    --name income-clf --candidate-version 2 \
    --metric f1 --min-improvement 0.01

```

Rellenar la Model Card

Abre `labs/lab5_e2e/MODEL_CARD_template.md` y completa cada apartado para `income-clf`. Las respuestas tipo:

- **Propósito.** Predecir si una persona gana más de 50K USD/año a partir de variables demográficas y laborales del censo US 1994.
- **Datos.** UCI Adult Income. ~30.000 registros. 14 variables. Licencia: dominio público.
- **Métricas globales.** F1 $\approx$ 0.66 en test holdout 20%.
- **Métricas por subgrupo.**
 - F1 sex=Female: ~0.55.
 - F1 sex=Male: ~0.70.
 - Disparidad significativa, atribuible a sesgo histórico del dataset.
- **Limitaciones.** Dataset de 1994. Sesgos conocidos en raza, género y origen. No representa la población actual.
- **Contraindicaciones.** NO usar para scoring de crédito real. NO usar para selección de personal. Es un dataset didáctico.
- **Mantenedor.** Tú.
- **Versión.** v1, fecha, commit hash.

Checklist de governance

Para `income-clf`:

- ☐ **Model Card** redactada.
- ☐ **Owner** asignado y on-call documentado.

- ☐ **Métricas por subgrupo** calculadas (F1 por sex, race).
- ☐ **Privacidad:** ¿qué campos son PII? Si los hay, ¿están anonimizados?
- ☐ **DPIA** redactada si afecta a decisiones automatizadas (RGPD Art. 35).
- ☐ **Decisión humana** posible: ¿existe la vía para que un humano revierta la predicción?
- ☐ **Audit log:** ¿podrías decir qué predijo el modelo el día X para el cliente Y?
- ☐ **Rollback:** ¿qué versión recuperarías si la actual rompe?
- ☐ **Disaster Recovery:** ¿qué pasa si MinIO se cae con todos los runs?
- ☐ **Costes:** ¿qué cuesta entrenar y servir al mes?

Discusión

Preguntas para reflexionar (individual o en grupo):

1. ¿Qué falta para que esto sea producción “real” en tu sector?
2. ¿Cómo gestionas secrets en CI hoy? ¿Cómo deberías?
3. ¿Dónde vive el modelo realmente en tu organización? (Registry, S3, K8s, Edge).
4. Si el modelo se equivoca y un cliente reclama, ¿cómo trazas la decisión?
5. ¿Cuál sería tu primer paso el lunes en tu trabajo?

7. Hoja de ruta de aprendizaje

Nivel	Skills
Junior MLOps	Docker, Git, Python, MLflow básico, FastAPI, CI básico
Mid	Kubernetes, Terraform, una feature store, Prometheus + Grafana + alerting, un orquestador (Airflow o Prefect)
Senior	Diseño de plataforma ML completa, multi-cloud, optimización de costes, governance + cumplimiento, multitenancy, edge ML, LLMOps

Pasar de junior a mid suele llevar 1-2 años de práctica seria. De mid a senior, otros 2-3 años.

8. Bibliografía y recursos

Libros

- *Designing Machine Learning Systems*, Chip Huyen (O'Reilly, 2022). El más completo y actual.
- *Practical MLOps*, Noah Gift (O'Reilly, 2021). Muy hands-on.
- *Machine Learning Engineering*, Andriy Burkov. Conciso y excelente.
- *Reliable Machine Learning*, Cathy Chen et al. (O'Reilly). SRE aplicado a ML.

Webs

- **ml-ops.org**: hoja de ruta de la comunidad. Imprescindible.
- **MLOps Community**: Slack muy activo y blog de calidad.
- **Made With ML** (<https://madewithml.com>): tutoriales gratuitos.
- **r/mlops** en Reddit.

Whitepapers

- Sculley et al. *Hidden Technical Debt in ML Systems* (2015).
- Google. *MLOps: Continuous delivery and automation pipelines in ML*.
- Mitchell et al. *Model Cards for Model Reporting* (2019).

9. Entregable opcional del curso

Un PR a tu repo personal con:

- El pipeline de datos versionado funcionando.
- Tres experimentos comparados en MLflow.
- Un modelo registrado y promocionado a Staging o Production.
- La API servida y probada.
- Un workflow CI verde (puede ser solo lint+test).
- La Model Card en Markdown.

No es obligatorio. Pero quien lo hace, suele sacar oferta de trabajo en MLOps en 6 meses. Está demostrado.

Lab 5 — Caso end-to-end + Governance

Tiempo estimado: **45 min** (30 min ejecución + 15 min discusión)

Objetivo

Recorrer el ciclo completo en una sola sesión y dejar evidencia de cada etapa. Luego, en grupo, identificar qué **falta** para una producción “real” más allá de un MVP didáctico.

El recorrido en 8 comandos

```
# 1. dependencias y entorno
docker compose -f docker/docker-compose.yml up -d
pip install -r requirements.txt

# 2. datos
cd labs/lab1_dataops
dvc pull
dvc repro

# 3. tests
cd ../..
pytest -q

# 4. entrenamiento + tracking
python labs/lab2_mlflow_dvc/src/train.py --model rf

# 5. promoción
python labs/lab2_mlflow_dvc/src/register_best.py \
    --experiment income-classifier --metric f1 --name income-clf
python labs/lab2_mlflow_dvc/src/promote.py \
    --name income-clf --version 1 --stage Staging

# 6. servir
docker build -t anban/income-api:0.1 -f labs/lab3_serving/Dockerfile labs/lab3_serving

docker run -d --network anban_default -p 8000:8000 \
    -e MLFLOW_TRACKING_URI=http://mlflow:5000 \
    anban/income-api:0.1

# 7. drift
python labs/lab4_cicd_monitoring/synthetic/make_drift.py
python labs/lab4_cicd_monitoring/src/drift_report.py \
    --reference labs/lab1_dataops/data/processed/test.parquet \
    --current labs/lab4_cicd_monitoring/data/processed/drifted.parquet \
    --output reports/drift.html

# 8. ¿supera al actual?
```

```
python labs/lab4_cicd_monitoring/src/promote_if_better.py \
  --name income-clf --metric f1 --min-improvement 0.01
```

Checklist de governance (rellenar en grupo)

Para el modelo `income-clf` que acabamos de poner en Staging:

- ☐ **Model Card** redactada: propósito, datos, métricas, sesgos, contraindicaciones.
- ☐ **Owner** asignado y on-call documentado.
- ☐ **Métricas por subgrupo**: F1 por sex, race; ¿hay disparidad?
- ☐ **Privacidad**: ¿qué campos son PII? ¿anonimización adecuada?
- ☐ **DPIA** si afecta a decisiones automatizadas (RGPD Art. 35).
- ☐ **Decisión humana** posible si el modelo se usa como input.
- ☐ **Audit log**: ¿se puede saber qué predijo este modelo el día X?
- ☐ **Rollback**: ¿qué versión vuelve si la actual rompe?
- ☐ **Disaster Recovery**: ¿qué pasa si MinIO se cae con todos los runs?
- ☐ **Costes**: ¿qué cuesta entrenar y qué cuesta servir al mes?

Discusión guiada (15 min)

Preguntas para el grupo:

1. ¿Qué falta para que esto sea producción “real” según el sector del alumno?
2. ¿Cómo gestionas secrets en CI? (no commitar nunca, GitHub Encrypted Secrets, Vault)
3. ¿Dónde vive el modelo realmente? (Registry, S3, K8s, Edge?)
4. Si el modelo se equivoca y un cliente reclama, ¿cómo trazas la decisión?
5. ¿Cuál sería el primer paso en tu organización el lunes?

Entregable del curso

Un PR a tu repo personal con:

- el pipeline de datos versionado,
- 3 experimentos comparados,
- el modelo en el Registry,
- la API funcionando,
- un workflow CI verde,
- la Model Card redactada en Markdown.

Guía paso a paso de los labs con explicaciones

En cada lab vas a teclear varios comandos. Esta guía acompaña cada uno de esos comandos con **qué hace, por qué lo necesitamos y qué deberías ver**. Léela en paralelo al README del lab cuando te pierdas. No sustituye al README; lo complementa.

Lab 1 — DataOps con DVC + Great Expectations

Pregunta que respondemos: ¿cómo garantizo que el dato que entra al modelo de hoy es el mismo que entró ayer (o uno mejor, controlado)?

Fase 0 · Levantar el entorno

```
docker compose -f docker/docker-compose.yml up -d
```

- **Qué hace:** arranca los cinco contenedores del curso. En el Lab 1 nos importan MinIO (donde DVC dejará el dataset) y Postgres (aunque no lo usemos directamente aquí).
- **Por qué:** sin MinIO funcionando no hay sitio donde subir el dataset versionado.
- **Comprueba:** `docker compose ps`. MinIO y Postgres deben estar `Up (healthy)`.

Fase 1 · Inicializar DVC

```
git init # solo si el repo no tiene git ya
dvc init
```

- **Qué hace** `dvc init`: crea el directorio `.dvc/` con la configuración del proyecto. A partir de ese momento DVC sabe que esta carpeta es un proyecto de versionado de datos.
- **Por qué:** DVC vive sobre Git. Necesita un repo Git inicializado para colocar sus archivos puntero (`.dvc`).
- **Comprueba:** debe aparecer la carpeta `.dvc/` con `config`, `.gitignore` y `cache/` (vacía).

Fase 2 · Configurar el remote (MinIO)

```
dvc remote add -d minio s3://datasets/lab1
dvc remote modify minio endpointurl http://localhost:9000
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345
```

- **Línea 1:** registra un *remote* llamado `minio` que apunta al bucket `datasets`, ruta `lab1`. `-d` lo marca como default (para que `dvc push` sin argumentos lo use).

- **Línea 2:** el endpoint S3 (binario, **9000**, no **9001** que es la consola).
- **Líneas 3 y 4:** credenciales. El `--local` guarda en `.dvc/config.local`, que **no** se commitea (está en `.gitignore`) para que tus secretos no acaben en GitHub.
- **Comprueba:** `cat .dvc/config` y `cat .dvc/config.local` deben mostrar lo que acabas de configurar.

Fase 3 · Descargar el dataset

```
mkdir -p data/raw
curl -L https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data \
-o data/raw/adult.csv
```

- **Qué hace:** baja el dataset **UCI Adult Income** (~3.9 MB), un clásico didáctico: predice si un adulto gana más o menos de 50k al año según edad, educación, horas trabajadas, etc.
- **Por qué:** necesitamos un dato real para versionar. Si la red de la sala va mal, ya tienes una copia en `datasets/adult/adult.csv`.

Fase 4 · Versionar el dataset con DVC

```
dvc add data/raw/adult.csv
git add data/raw/adult.csv.dvc data/raw/.gitignore
git commit -m "track dataset with dvc"
dvc push
```

- `dvc add`: calcula el hash MD5 del CSV, mueve el archivo al cache de DVC y crea `adult.csv.dvc` con el hash. También añade `adult.csv` al `.gitignore` para que **el CSV pesado no se commitee**.
- `git add` + `git commit`: versionamos el *puntero*, no el archivo real.
- `dvc push`: sube el blob real (el CSV) a MinIO.
- **Comprueba:** abre la consola de MinIO (`http://localhost:9001`) y navega a `datasets/lab1/files/md5/...`. Debe haber un archivo con un nombre que es el hash.

Concepto clave: *Git versiona código y punteros. DVC versiona blobs en almacenamiento externo. Juntos te dan reproducibilidad sin saturar Git.*

Fase 5 · Validar el dataset con Great Expectations

```
python src/ge_validate.py data/raw/adult.csv
```

- **Qué hace** `ge_validate.py`: carga el CSV en un DataFrame y ejecuta cuatro expectations: schema, rango de edades, dominio de `workclass`, no-nulos en `income`.
- **Por qué:** queremos detectar datos malos **antes** de que el modelo entrene con ellos. Es un *fail-fast*.

- **Comprueba:** salida `ALL EXPECTATIONS PASSED` . Si alguna falla, el script termina con código distinto de 0 y el pipeline se para (lo verás en la fase 7).

Fase 6 · Definir y ejecutar el pipeline

```
dvc repro
```

- **Qué hace:** lee `dvc.yaml` , calcula el grafo de stages (`validate` → `preprocess`), compara hashes de entradas con la última ejecución, y solo re-ejecuta lo que ha cambiado.
- **Por qué:** así no entrenas con datos sin validar, y no preprocesas dos veces lo mismo.
- **Comprueba:** aparecen `data/processed/train.parquet` y `test.parquet` . `dvc.lock` se actualiza con los hashes de los outputs.

Fase 7 · Romper el dato a propósito

```
sed -i '5s/.*/16, "??"/' data/raw/adult.csv  
dvc repro
```

- **Qué hace:** mete una fila con `age = 16` (fuera del rango 17–90 que validamos).
- **Resultado esperado:** `validate` falla, `preprocess` no se ejecuta. Eso es lo que queremos: la validación **es un gate**.

Recupera:

```
dvc checkout data/raw/adult.csv      # restaura desde el cache  
dvc repro
```

Fase 8 · Cambia el código

Edita `src/preprocess.py` y cambia `test_size=0.2` a `0.3` . Ejecuta:

```
dvc repro
```

- **Qué pasa:** `validate` no se reejecuta (sus inputs no han cambiado), `preprocess` sí (cambió su código).
- **Por qué importa:** DVC entiende qué dependencia cambió y rehace solo lo afectado. En un pipeline grande eso significa minutos en lugar de horas.

Lab 2 — MLflow Tracking + Model Registry

Pregunta que respondemos: ¿cómo registro cada entrenamiento de forma reproducible y promocio el mejor modelo de forma controlada?

Fase 1 · Comprobar el servidor MLflow

```
curl -s http://localhost:5050/ -o /dev/null -w "%{http_code}\n"
```

- Debe devolver `200`. Si no, mira logs: `docker logs anban-mlflow --tail=30`.
- **Por qué:** todo el lab se apoya en el servidor de tracking. Si no responde, los `mlflow.log_*` fallarán o se irán a una carpeta local (lo último que quieres).

Fase 2 · Configurar el cliente en el notebook

```
import os, mlflow
os.environ["MLFLOW_TRACKING_URI"] = "http://localhost:5050"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://localhost:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

mlflow.set_tracking_uri("http://localhost:5050")
mlflow.set_experiment("adult-income")
```

- **Por qué tantas variables:** MLflow registra los metadatos en Postgres (a través del servidor) y los artefactos (modelo, gráficas) en MinIO. Sin las credenciales de MinIO no puede subir el modelo.
- **Comprueba:** en la UI de MLflow aparece el experimento "adult-income" (al principio vacío).

Fase 3 · Primer run

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score
import mlflow, mlflow.sklearn

with mlflow.start_run(run_name="rf-baseline"):
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)

    model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)

    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    f1 = f1_score(y_test, pred)

    mlflow.log_metric("f1", f1)
    mlflow.sklearn.log_model(model, "model", input_example=X_train.iloc[:3])
```

Qué hace cada línea:

- `start_run`: abre un run nuevo. Todo lo que loggees dentro queda asociado a ese run.
- `log_param`: parámetros que decidiste tú (hiperparámetros, configuración).
- `log_metric`: métricas medibles del experimento. Pueden ser varias y MLflow las trackea como serie temporal.

- `log_model` : serializa el modelo y lo sube al artifact store (MinIO). El `input_example` ayuda a futuros usuarios a saber qué forma de input espera.

Comprueba: ve a la UI de MLflow. Tu run aparece con sus params, su F1 y un directorio `model/` con el `.pkl` , `MLmodel` (descriptor) y `conda.yaml` (entorno).

Fase 4 · Comparar runs

Lanza el mismo notebook con `n_estimators=300` , `max_depth=12` . Vuelve a la UI:

- Marca los dos runs y pulsa **Compare**.
- Verás una tabla con sus params, métricas y un gráfico de paralelas para ver visualmente qué configuración fue mejor.

Concepto: *un experiment es la pregunta ("¿qué configuración predice mejor el income?"). Cada run es una respuesta concreta. Comparar runs es el día a día del data scientist.*

Fase 5 · Registrar el modelo

En la UI, abre el run ganador, despliega `model` y pulsa **Register Model**. Le pones nombre `adult-income-clf` .

Equivalente programático:

```
result = mlflow.register_model(
    f"runs:{run_id}/model",
    "adult-income-clf"
)
```

Eso crea la versión 1 del modelo en el **Registry**.

Fase 6 · Promocionar a Staging

En la pestaña **Models** del Registry, sobre la versión 1, cambias el stage a `Staging` .

- **Por qué:** el Lab 3 cargará el modelo desde `models:/adult-income-clf/Staging` . Así desacoplas el **identificador del modelo** del **identificador del run**. Si mañana entrenas un modelo mejor, basta con promocionar la nueva versión a Staging y el Lab 3 lo recoge sin cambiar una línea.

Concepto: *Registry es el catálogo central. Los stages (`None` , `Staging` , `Production` , `Archived`) son el ciclo de vida controlado del modelo, equivalente al ciclo de releases del software.*

Lab 3 — Servir el modelo con FastAPI + Docker

Pregunta que respondemos: ¿cómo convierto el modelo registrado en algo que cualquier sistema pueda consumir por HTTP, empaquetado para que funcione en cualquier máquina?

Fase 1 · Esqueleto FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import mlflow.pyfunc, os

os.environ["MLFLOW_TRACKING_URI"] = "http://mlflow:5000"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://minio:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

app = FastAPI(title="Adult Income API")
model = mlflow.pyfunc.load_model("models:/adult-income-clf/Staging")

class Input(BaseModel):
    age: int
    workclass: str
    education_num: int
    hours_per_week: int

@app.get("/health")
def health(): return {"status": "ok"}

@app.get("/version")
def version(): return {"model": "adult-income-clf", "stage": "Staging"}

@app.post("/predict")
def predict(payload: Input):
    pred = model.predict([payload.dict()])
    return {"prediction": int(pred[0])}
```

Qué hace cada bloque:

- **Variables de entorno:** decirle al cliente MLflow dónde está el tracking server y el bucket. Importante: usamos los nombres de servicio Docker (`mlflow` , `minio`), no `localhost` — la API correrá dentro de la red Docker.
- `load_model("models:/.../Staging")` : descarga la versión actualmente en Staging. Si mañana promocionas otra versión, basta con reiniciar el contenedor para recogerla.
- `/health` : endpoint que devuelve 200 si el servicio está vivo. Lo usan orquestadores (Kubernetes, Docker) para detectar contenedores rotos.
- `/version` : debugging humano y trazabilidad. Si dos servicios devuelven predicciones distintas, primero consultas `/version` para ver si están sirviendo el mismo modelo.
- `/predict` : el endpoint principal. Pydantic valida el payload **antes** de invocar al modelo.

Fase 2 · Probar en local sin Docker

```
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
```

```
-d '{"age":39,"workclass":"Private","education_num":13,"hours_per_week":40}'
```

- **Por qué probarlo así antes:** Docker añade complejidad (red, build, paths). Si la app funciona ya en local, sabemos que el problema en Docker (si surge) está en el empaquetado, no en el código.

Fase 3 · Dockerfile multi-stage

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
COPY app.py .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

- **Multi-stage:** la primera etapa instala dependencias en `/root/.local`. La segunda etapa parte de una imagen limpia y copia solo los paquetes — sin caches de pip, sin código de build. Resultado: imagen final mucho más pequeña.
- **Por qué importa el tamaño:** una imagen de 3 GB tarda en bajar a cada nodo del cluster. Una de 200 MB se despliega en segundos.
- **Comprueba:** `docker images mi-api` debe mostrar un tamaño razonable (~300 MB para esta API).

Fase 4 · Construir y ejecutar

```
docker build -t adult-income-api:0.1 .
docker run --rm -p 8000:8000 --network anban_default adult-income-api:0.1
```

- `--network anban_default`: conectamos el contenedor a la red de Docker que ya creó el `docker-compose`. Así puede ver a `mlflow` y `minio` por nombre.
- **Comprueba:** `curl http://localhost:8000/health` devuelve `{"status":"ok"}`.

Fase 5 · Benchmark con Locust

```
# locustfile.py
from locust import HttpUser, task

class APIUser(HttpUser):
    @task
    def predict(self):
        self.client.post("/predict", json={
            "age": 39, "workclass": "Private",
```

```
        "education_num": 13, "hours_per_week": 40
    })
```

```
locust -f locustfile.py --headless -u 50 -r 5 -t 1m --host http://localhost:8000
```

- **Qué mide:** 50 usuarios simultáneos durante 1 minuto, ramping up a 5 usuarios/segundo.
- **Métricas que importan:**
 - **p50:** la mitad de las peticiones tardan menos que esto.
 - **p95 / p99:** las “colas” de la distribución. Una p99 alta significa que algunos usuarios sufren picos de latencia.
 - **Requests/s (throughput):** capacidad del servicio.
- **Objetivo del lab:** p95 < 100 ms. Si lo superas, mira:
 1. ¿El modelo está cargado **una vez al iniciar** o se carga en cada petición? Si es lo segundo, mátalos: muévelo fuera del handler.
 2. ¿Tienes suficientes workers? `uvicorn --workers 4`.

Lab 4 — CI/CD con GitHub Actions + Monitoring con Evidently

Pregunta que respondemos: ¿cómo automatizo el ciclo entrenar-evaluar-promover y cómo detecto que el modelo se está desgastando?

Fase 1 · Workflow base

Crea `.github/workflows/ml.yml`:

```
name: ml-pipeline
on: [push, pull_request]

jobs:
  pipeline:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env: { POSTGRES_USER: mlflow, POSTGRES_PASSWORD: mlflow, POSTGRES_DB: mlflow }
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - run: pip install pytest ruff
      - run: ruff check src/ # lint
      - run: pytest -q # tests
      - run: python src/train.py # entrena
```

```
- run: python src/evaluate.py --baseline-stage Production # compara
- run: python src/promote.py # promociona si mejora
```

Qué hace cada paso:

- **ruff** : linter Python ultra-rápido. Detecta código mal indentado, imports no usados, errores tipográficos típicos.
- **pytest** : ejecuta tests unitarios. Si un test falla, el pipeline se corta — no entrenamos sobre código roto.
- **train.py** : entrena un nuevo modelo. Loggea params/métricas en MLflow (idealmente apuntando a un servidor de tracking real, no al efímero del CI).
- **evaluate.py** : carga el modelo actual en **Production** y compara F1. Si el nuevo no mejora, **exit 1**.
- **promote.py** : si pasó evaluate, registra y promociona la nueva versión.

Concepto: los pasos están en orden de coste. Lo barato (lint, tests) va primero porque si rompen te enteras pronto y no gastas minutos entrenando.

Fase 2 · Estrategia de despliegue

Estrategia	Cómo funciona	Cuándo usarla
Blue-green	Levantas la versión nueva (green) al lado de la vieja (blue). Cambias el router de golpe.	Cambios importantes. Rollback instantáneo.
Canary	La versión nueva recibe el 5 % del tráfico al principio; vas subiendo.	Quieres detectar problemas con tráfico real sin arriesgar todos los usuarios.
Shadow	La versión nueva recibe el 100 % del tráfico en paralelo pero sus respuestas no se devuelven. Comparas predicciones.	Validación previa sin riesgo cero.

Fase 3 · Informe Evidently

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, ClassificationPreset

ref = pd.read_csv("data/train_reference.csv") # el que usaste para entrenar

current = pd.read_csv("data/last_week_inference.csv") # un lote reciente

report = Report(metrics=[DataDriftPreset(), ClassificationPreset()])
report.run(reference_data=ref, current_data=current)
report.save_html("monitoring/drift_report.html")
report.save_json("monitoring/drift_report.json")
```

- **DataDriftPreset** : detecta movimiento de las distribuciones de features.

- `ClassificationPreset` : mide degradación de las métricas del modelo en el nuevo lote (si tienes ground truth).
- **JSON + HTML**: el JSON va a un endpoint `/monitor` que sirve el resumen al equipo. El HTML va al equipo de producto para ver gráficas.

Fase 4 · Inyectar drift y observarlo

```
import numpy as np
current_drifted = current.copy()
current_drifted["age"] += np.random.randint(10, 20, size=len(current)) # envejecemos

current_drifted["hours_per_week"] *= 1.5
```

Vuelves a correr el informe. Ahora deberías ver:

- Tests PSI / KS sobre `age` y `hours_per_week` por encima del umbral.
- En la sección de drift, esas columnas marcadas en rojo.

Concepto: los tests estadísticos (PSI, KS, Chi², Wasserstein) ponen un **número objetivo** sobre algo que de otra forma sería ojímetro. PSI > 0.2 es un umbral común para “esto se ha movido lo suficiente como para preocuparse”.

Lab 5 — Caso integrador end-to-end + Governance

Pregunta que respondemos: ¿qué pinta tiene el ciclo entero ejecutado de punta a punta, y qué falta para producción real?

Fase 1 · Recorrer el pipeline completo

```
dvc pull                # baja datos del Lab 1
pytest                  # valida que nada se ha roto
python src/train.py     # entrena, logea en MLflow
python src/evaluate.py  # compara contra Production
python src/promote.py   # promociona si mejora
docker build -t adult-income-api:${git rev-parse --short HEAD} .
docker run -d -p 8000:8000 --network anban_default adult-income-api:${git rev-parse --short HEAD}
curl http://localhost:8000/predict ...
```

Lo importante no es que **cada comando** funcione (ya los conoces) sino que **el orden** es el orden.

Fase 2 · Model card

Crea `MODEL_CARD.md` con:

- **Intended use:** para qué sirve (predicción binaria de income > 50k).

- **Out of scope:** para qué **no** sirve (no es válido para tomar decisiones sobre crédito sin revisión humana; los datos son de EE.UU. de los 90, no extrapolan).
- **Datos de entrenamiento:** UCI Adult Income, ~30k filas, sesgo demográfico conocido.
- **Métricas globales y por subgrupo:** accuracy y F1 desglosados por `race`, `sex`, `native-country`. Importante: muchas veces el modelo funciona “bien” globalmente pero mucho peor para un subgrupo concreto.
- **Limitaciones conocidas:** clases desbalanceadas, label originalmente categórica.
- **Cambios entre versiones:** changelog.

Fase 3 · Discusión guiada: qué falta para producción real

Lista para reflexionar (sin código):

1. **Secretos:** ¿dónde guardas las credenciales de MinIO/Postgres en producción? (Spoiler: no en `.env` commiteado. AWS Secrets Manager, Vault, Doppler).
2. **Observabilidad:** ¿quién se entera si tu API responde 500 ahora mismo? (Prometheus + Grafana, Sentry, Datadog).
3. **Rollback:** si el modelo nuevo da peores predicciones que el viejo, ¿cuánto tardas en volver atrás? (Idealmente: 30 segundos vía Registry).
4. **Autoscaling:** si llegan 10x peticiones, ¿se cae o escala? (Kubernetes HPA, ECS Auto Scaling).
5. **Alertas de drift:** ¿quién mira el informe Evidently? (Cron diario que postea en Slack si PSI > umbral).
6. **Reentrenamiento:** ¿cada cuánto reentrenas? ¿con qué datos? (CT — Continuous Training, igual de importante que CI/CD).
7. **Compliance:** si un usuario afectado por una predicción pide explicación, ¿puedes darla? (SHAP, LIME, audit logs).
8. **Costes:** ¿cuánto cuesta entrenar y servir? ¿se justifica? (FinOps en ML: GPUs son caras, batch suele ser 100x más barato que online).
9. **Versionado conjunto código-datos-modelo:** ¿puedes reconstruir el modelo de hace 6 meses bit a bit? Si no, falla auditoría.
10. **Fairness:** ¿el modelo discrimina sistemáticamente a un grupo? ¿cómo lo mides?

Lo que sí tienes ya: versionado de datos, validación, tracking, registry, API empaquetada, CI/CD, monitoring de drift. Eso te coloca en **nivel 1–2 de madurez** de la escala de Google. La mayoría de empresas están en nivel 0. Has dado un salto enorme en 2 días.

Esto es una chuleta de uso. Toda la información operativa que necesitas durante los dos días en un solo sitio. Imprime esta hoja si lo prefieres.

Stack del curso

```
# Levantar
docker compose -f docker/docker-compose.yml up -d

# Ver estado
docker compose -f docker/docker-compose.yml ps

# Logs de un servicio
docker compose -f docker/docker-compose.yml logs -f mlflow

# Parar todo
docker compose -f docker/docker-compose.yml down

# Limpiar volúmenes (cuidado, borra datos)
docker compose -f docker/docker-compose.yml down -v
```

URLs y credenciales

Servicio	URL	Usuario / Pass
MLflow Tracking	http://localhost:5000	—
MinIO Console	http://localhost:9001	minio / minio12345
MinIO API	http://localhost:9000	minio / minio12345
Postgres	localhost:5432	mlflow / mlflow
JupyterLab	http://localhost:8888	token <code>anban</code>
Income API	http://localhost:8000	—
Swagger API	http://localhost:8000/docs	—

Variables de entorno (ponlas en cada terminal nueva)

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
export MODEL_URI=models:/income-clf/Staging
```

Truco: guarda esto en `setenv.sh` y haz `source setenv.sh` .

DVC

```
# Inicializar
dvc init

# Configurar remoto MinIO
dvc remote add -d minio s3://datasets/lab1
dvc remote modify minio endpointurl http://localhost:9000
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345

# Trackear un fichero
dvc add data/raw/adult.csv

# Subir al remoto
dvc push

# Recuperar desde el remoto
dvc pull

# Ejecutar el pipeline
dvc repro

# Ver el grafo del pipeline
dvc dag

# Ver estado
dvc status

# Recuperar un fichero a su versión correcta
dvc checkout data/raw/adult.csv
```

Git + DVC

```
# Commit típico tras dvc add  
git add data/raw/adult.csv.dvc data/raw/.gitignore  
git commit -m "trackea dataset"  
  
# Commit típico tras editar el pipeline  
git add dvc.yaml dvc.lock params.yaml  
git commit -m "ajusta test_size"
```

MLflow

```
# UI
open http://localhost:5000

# Listar experimentos
mlflow experiments search

# Listar runs
mlflow runs list --experiment-id 1
```

Desde Python

```
import mlflow, os
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("income-classifier")

with mlflow.start_run(run_name="rf"):
    mlflow.log_params({"n_estimators": 300})
    mlflow.log_metrics({"f1": 0.78})
    mlflow.set_tag("owner", "yo")
    mlflow.sklearn.log_model(model, name="model")
```

Model Registry

```
import mlflow
client = mlflow.MlflowClient()

# Registrar
mv = client.create_model_version(
    name="income-clf",
    source=f"runs:{run_id}/model",
    run_id=run_id,
)

# Promocionar
client.transition_model_version_stage(
    name="income-clf",
    version=1,
    stage="Staging",
)

# Cargar
m = mlflow.pyfunc.load_model("models:income-clf/Staging")
```


FastAPI + Docker

```
# Local con autoreload
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Build imagen
docker build -t anban/income-api:0.1 -f Dockerfile .

# Run en Linux
docker run --rm -d --network host \
  -e MLFLOW_TRACKING_URI=http://localhost:5000 \
  -e MODEL_URI=models:/income-clf/Staging \
  anban/income-api:0.1

# Run en Mac/Windows
docker run --rm -d --network anban_default -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MODEL_URI=models:/income-clf/Staging \
  anban/income-api:0.1

# Ver logs
docker logs -f income-api

# Stop
docker stop income-api
```

Probar la API con curl

```
# Health
curl -s http://localhost:8000/health | jq

# Version
curl -s http://localhost:8000/version | jq

# Predict
curl -s -X POST http://localhost:8000/predict \
  -H 'content-type: application/json' \
  -d '{
    "age": 39, "workclass": "State-gov", "education_num": 13,
    "marital_status": "Never-married", "occupation": "Adm-clerical",
    "relationship": "Not-in-family", "race": "White", "sex": "Male",
    "capital_gain": 2174, "capital_loss": 0,
    "hours_per_week": 40, "native_country": "United-States"
  }' | jq

# Metrics
curl -s http://localhost:8000/metrics

# Monitor (drift)
curl -s http://localhost:8000/monitor | jq
```

Locust (stress test)

```
locust -f tests/locustfile.py --host http://localhost:8000 \  
--users 50 --spawn-rate 10 --run-time 30s --headless
```

Evidently y drift

```
# Generar dataset con drift
python synthetic/make_drift.py

# Reporte HTML
python src/drift_report.py \
  --reference ../lab1_dataops/data/processed/test.parquet \
  --current data/processed/drifted.parquet \
  --output reports/drift.html
```

Promote condicional

```
python src/promote_if_better.py \  
    --name income-clf \  
    --candidate-version 2 \  
    --metric f1 \  
    --min-improvement 0.01
```

Atajos para depuración

```
# Ver puertos ocupados
lsof -i :5000

# Reiniciar un servicio
docker compose -f docker/docker-compose.yml restart minio

# Entrar a un contenedor
docker exec -it anban-mlflow bash

# Ver red de Docker
docker network ls
docker network inspect anban_default
```

Si algo se rompe sin remedio

```
# Reset total del stack (pierdes runs)
docker compose -f docker/docker-compose.yml down -v
docker compose -f docker/docker-compose.yml up -d

# Reset del lab 1
cd labs/lab1_dataops
rm -rf data/processed
dvc repro

# Reentrenar y promocionar de cero
python labs/lab2_mlflow_dvc/src/train.py --model rf
python labs/lab2_mlflow_dvc/src/register_best.py \
    --experiment income-classifier --metric f1 --name income-clf
python labs/lab2_mlflow_dvc/src/promote.py \
    --name income-clf --version 1 --stage Staging
```

Errores frecuentes y cómo resolverlos

Esta sección recoge los problemas que aparecen casi en cada edición del curso. Si te encuentras con un error que no está aquí, búscalo por palabras clave en este capítulo antes de pedir ayuda. La mayoría de los fallos vienen de cuatro causas:

1. Puertos ocupados por otra cosa en tu máquina.
 2. Mezcla de entornos Python (sistema / venv / anaconda).
 3. Credenciales o endpoints S3 mal configurados.
 4. Confusión entre `localhost` (desde tu PC) y nombres de servicio dentro de Docker.
-

1 · Docker no arranca o no responde

1.1 `Cannot connect to the Docker daemon at unix:///var/run/docker.sock`

Significa que el motor de Docker no está corriendo.

- **Windows / macOS:** abre Docker Desktop manualmente y espera a que el icono se ponga verde.
- **Linux:**

```
sudo systemctl start docker
sudo systemctl enable docker    # para que arranque al inicio
```

1.2 `permission denied while trying to connect to the Docker daemon` (Linux)

Tu usuario no está en el grupo `docker` :

```
sudo usermod -aG docker $USER
newgrp docker
```

1.3 Docker Desktop “Engine stopped” en Windows

Reinicia WSL desde PowerShell:

```
wsl --shutdown
```

Después vuelve a abrir Docker Desktop.

2 · Puertos ocupados

2.1 `bind: address already in use`

Cuando arrancas el stack te dice que un puerto está en uso (5000, 5432, 8888, 9000 o 9001). Algún otro programa ya lo tiene tomado.

Identifica al culpable:

- Linux / macOS:

```
lsof -iTCP:5000 -sTCP:LISTEN
```

o:

```
ss -tlnp | grep 5000
```

- Windows (PowerShell):

```
Get-NetTCPConnection -LocalPort 5000 | Select-Object OwningProcess  
Get-Process -Id <PID>
```

Solución A — para el programa que lo usa. Por ejemplo, otro proyecto tuyo con el mismo puerto.

Solución B — cambia el puerto del host en el compose. Edita `docker/docker-compose.yml` y cambia el lado izquierdo del `:`:

```
mlflow:  
  ports:  
    - "5050:5000" # antes "5000:5000"
```

Aplica el cambio sin tocar el resto del stack:

```
docker compose up -d mlflow
```

*Cuidado: el lado **izquierdo** es el puerto en tu máquina (host). El **derecho** es el puerto dentro del contenedor y no debe cambiarse — está pegado a la configuración interna del servicio.*

2.2 Puerto 9000 vs 9001 (MinIO) — error muy frecuente

MinIO expone **dos puertos distintos** y se confunden constantemente:

Puerto	¿Qué es?	¿Quién lo usa?
9000	API S3 (binaria)	DVC, MLflow, <code>boto3</code> , <code>mc</code>
9001	Consola web HTML	Humanos abriendo el navegador

Si configuras DVC apuntando a `9001` verás errores `400 Bad Request` . Si abres `9000` en el navegador, verás “XML inválido” o similar — no es para el navegador.

Regla: **DVC y MLflow siempre 9000. Tu navegador siempre 9001.**

3 · Postgres: “esta página no funciona” en el navegador

Postgres **no es una página web**. No se abre desde el navegador, habla un protocolo binario propio.

Conecta con uno de estos clientes:

- **Línea de comandos:**

```
PGPASSWORD=mlflow psql -h localhost -p 5432 -U mlflow -d mlflow
```

- **DBeaver** (GUI cross-platform):

- Host: `localhost` · Port: `5432` · DB: `mlflow` · User: `mlflow` · Pass: `mlflow` .

- **Desde Jupyter:**

```
import psycopg2
conn = psycopg2.connect(host="postgres", port=5432, dbname="mlflow", user="mlflow",
password="mlflow")
```

Desde un contenedor, el host se llama `postgres` , no `localhost` .

4 · `localhost` vs nombres de servicio Docker

Desde dónde te conectas	A dónde	Host a usar	Puerto
Tu PC (host)	MLflow	<code>localhost</code>	<code>5050</code>
Tu PC (host)	MinIO S3	<code>localhost</code>	<code>9000</code>
Tu PC (host)	Postgres	<code>localhost</code>	<code>5432</code>
Contenedor (Jupyter, etc.)	MLflow	<code>mlflow</code>	<code>5000</code> (interno)
Contenedor	MinIO S3	<code>minio</code>	<code>9000</code>
Contenedor	Postgres	<code>postgres</code>	<code>5432</code>

Si confundes los dos lados, verás errores del tipo “Connection refused” o “Name or service not known”.

5 · Errores de Python y entornos virtuales

5.1 `ModuleNotFoundError: No module named 'X'`

El paquete no está instalado **en el Python que estás usando**. Comprueba qué Python ejecutas:

```
which python      # macOS/Linux/WSL
where python      # Windows
```

Y compara con `which dvc` / `which mlflow`. Si están en directorios distintos (típico: `dvc` en anaconda, tu venv en otra parte), tienes mezcla.

Solución general: activa el venv y ejecuta el paquete vía Python:

```
source .venv/bin/activate
python -m dvc push
```

Si quieres dejarlo bien:

```
pip install --force-reinstall dvc dvc-s3
hash -r      # zsh/bash: refresca cache de paths
which dvc    # debe estar dentro del .venv
```

5.2 `IndentationError: unexpected indent` en Jupyter

Has copiado código con espacios al principio de línea. Selecciona toda la celda (`Ctrl+A` dentro), pulsa `Shift+Tab` varias veces para sacar la indentación, y vuelve a ejecutar.

5.3 Anaconda interfiere con el venv

En el shebang del binario aparece `#!/home/usuario/anaconda3/bin/python` aunque tengas un venv activo. Causa: el comando está en `~/anaconda3/bin/`, que va antes en `$PATH`.

Soluciones:

- **Solución puntual:** `python -m <comando>` para forzar el Python del venv.
- **Solución permanente:** instala el comando dentro del venv (`pip install --force-reinstall <paquete>`) y limpia la caché del shell (`hash -r`).
- **Solución radical:** desactiva el auto-actívalo de anaconda:

```
conda config --set auto_activate_base false
```

6 · DVC: errores frecuentes

6.1 `remote 'X' doesn't exist`

Has escrito mal el nombre del remote (típico typo: `minio` en lugar de `minio`). Comprueba los existentes:

```
dvc remote list
```

6.2 `requires 'dvc-s3' to be installed`

Falta el plugin S3 para DVC:

```
pip install dvc-s3
```

Si dices que ya está instalado pero el error persiste, casi seguro estás ejecutando el `dvc` de otro Python (ver sección 5.3).

6.3 `Bad Request (400)` al hacer `dvc push` o `dvc pull`

Tres causas, en orden de probabilidad:

1. **Endpoint apuntando a 9001 en lugar de 9000.** Cámbialo:

```
dvc remote modify minio endpointurl http://localhost:9000
```

2. **Credenciales mal o ausentes.** Pon las del entorno del curso (no se commitean, van a `.dvc/config.local`):

```
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345
```

3. **MinIO no está arriba o sin bucket.** Comprueba:

```
curl -s http://localhost:9000/minio/health/live -o /dev/null -w "%{http_code}\n"
```

Debe devolver `200`. Y verifica buckets:

```
docker exec anban-minio mc alias set local http://localhost:9000 minio minio12345
docker exec anban-minio mc ls local/
```

6.4 Tras clonar un repo: `Unable to locate credentials`

`.dvc/config.local` está en `.gitignore` adrede (no se commitean secretos). En el clon nuevo configura las credenciales otra vez:

```
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345
dvc pull
```

7 · Jupyter

7.1 Pide token al entrar

Es lo normal. Introduce `anban` (el token está configurado en `docker-compose.yml`).

7.2 `Server connection closed` o “no se puede conectar”

El contenedor de Jupyter está parado. Comprueba:

```
docker compose ps jupyter
docker logs anban-jupyter --tail=30
```

Si está `Exited`, reinícialo:

```
docker compose start jupyter
```

7.3 Falta un paquete en Jupyter

Instálalo desde una celda:

```
!pip install psycopg2-binary
```

Esto solo dura mientras el contenedor exista. Para hacerlo permanente, añádelo al `command:` del servicio Jupyter en el `docker-compose.yml`.

8 · MLflow no responde o se queda colgado

La primera vez tras un `up`, MLflow instala dependencias al arrancar (`psycopg2-binary` , `boto3`). Tarda 30–60 segundos. Mira los logs:

```
docker logs anban-mlflow --tail=20 -f
```

Cuando veas `Listening at: http://0.0.0.0:5000`, está listo.

9 · Comandos universales para diagnosticar

Guárdate estos: resuelven el 80 % de los problemas.

```
# ¿Qué contenedores hay y en qué estado?
docker compose ps

# ¿Qué dice el contenedor X?
docker logs <nombre> --tail=50

# ¿Mi puerto está libre? (Linux/macOS)
lsof -iTCP:<puerto> -sTCP:LISTEN

# ¿MinIO API responde?
curl -s http://localhost:9000/minio/health/live -o /dev/null -w "%{http_code}
\n"

# Reiniciar un servicio sin tocar el resto
docker compose restart <servicio>

# Empezar de cero (pierdes datos en volúmenes)
docker compose down -v
docker compose up -d
```

10 · Cuándo pedir ayuda

Antes de levantar la mano en clase o escribir al foro, ten preparado:

1. **Qué intentabas hacer** (paso del lab, comando exacto).
2. **Qué error sale** (copia-pegar completo, no resumido).
3. **Qué has comprobado** (al menos `docker compose ps` y los logs del contenedor en cuestión).

Con esos tres datos, el 99 % de los errores se resuelve en un minuto.